



SAS[®] Viya[®] Platform: Troubleshooting

2024.09-2026.01

This document might apply to additional versions of the software. Open this document in [SAS Help Center](#) and click on the version in the banner to see all available versions.

Introduction	2
Overview	2
Use Commands or Dashboards	3
General Areas to Review for Problems	3
Client Interfaces	3
Services and Servers	4
Infrastructure	5
First Steps	5
Verify That Pods Are Ready	5
Determine When the Problem Occurred	7
Check Kubernetes Events	7
Review Metric Data	7
Troubleshoot Pod Failures	8
Before You Start Troubleshooting	8
Look for Symptoms	8
Identify the Pods That Are Not Running	9
Investigate Why the Pod Is Not Running	12
Resolve the Problem	16
Troubleshoot a Failure to Start Compute Server	21
Start Here	21
Determine How the Compute Server Pod Was Created	22
Examine the Compute Service Logs	24
Examine the Launcher Service Logs	27

Investigate a Failure Related to SAS Workload Orchestrator	28
Investigate a Failure Related to SAS Launcher Service	32
Increase Launch Time-Out Period	35
Monitoring SAS CAS Server	37
About the CAS Server	37
SAS CAS Server Architecture	37
Workload Placement Strategy	38
Describe Your SAS Viya Platform Deployment	40
Introduction	40
About Elevated Kubernetes Permissions	40
Set the \$viya Environment Variable	40
SAS Viya Deployment Release	41
Nodes in Your Kubernetes Cluster	41
SAS Viya Namespaces	44
SAS Viya Pods Overview	44
SAS Viya Pods: Stateful Services	46
SAS Viya Pods: Stateless Services	48
SAS Viya Pods: Programming Run-time Servers	50
SAS Viya Pods: Cloud Analytic Server (CAS)	51
Persistent Storage for SAS Viya Software	54
Avoid Database Issues	55
Troubleshoot Command-Line Interface Issues	56
Correct a Custom Script Failure	57
Rebalance Pods after Node Failure in HA Environment	57
Recover from a Disaster	58
Feature-Specific Troubleshooting Information	58
Get Support	59
Before You Contact Technical Support	59
SAS Technical Support Services and Policies	59
Reference	60
About the Current Namespace	60
About Kubectl Command Output	60
About Compute Server and Job Execution Service Job Names	60
Scenarios When Compute Server Is Not Created	61

Introduction

Overview

This document provides the following information:

- Strategies to use when troubleshooting the SAS Viya platform.

- One location from which to access all feature-level troubleshooting topics for the SAS Viya platform.
- Access to tools and processes that ensure when you contact SAS technical support, you can provide required information immediately.

Use Commands or Dashboards

There are two general ways to view information about your SAS Viya platform deployment:

kubectll commands

These commands can show you information that is valid for a moment in time.

Applications that show log and metric information via dashboards

Dashboard applications can show you information related to a specified period of time. This approach has the advantage of enabling you to see only the changes that occur during that time period.

Your organization might have a preferred application. If not, consider using the SAS Viya Monitoring for Kubernetes logging and monitoring tools.

SAS Viya Monitoring for Kubernetes provides log- and metric-monitoring tools, including components that are designed specifically for SAS Viya 4 services, components, and applications. Using these tools, you can implement the following solutions:

- Collect, aggregate, and display log messages and information from throughout your environment.
- Collect, view, and manage metrics, as well as send alerts from throughout your environment.

You can deploy these tools, without charge, from the [SAS Viya Monitoring for Kubernetes](#) GitHub site.

These tools are intended for Kubernetes and SAS administrators. See the [SAS Viya Monitoring for Kubernetes](#) documentation.

General Areas to Review for Problems

Typically, problems in SAS Viya platform occur in one of these areas.

Client Interfaces

Web Applications

Problems in SAS web applications (such as SAS Environment Manager or SAS Studio) can appear as error messages or an application failure (hang).

Many of these problems require assistance from SAS Technical Support.

IMPORTANT Before you contact SAS Technical Support, gather troubleshooting information about these problems. For instructions, see [“Before You Contact Technical Support” on page 59](#).

Command-Line Interface

For issues with the `sas-viya` command-line interface and its plug-ins, see [“Troubleshoot Command-Line Interface Issues” on page 56](#).

For syntax details, see the CLI integrated help:

```
sas-viya help
```

REST APIs

When troubleshooting problems in REST API implementations, review the log files that are generated by the service that is called by the affected REST API. See [“Services and Servers” on page 4](#).

Services and Servers

Problems in the services and servers appear through HTTP 500-type responses to requests.

You can view these responses in the following ways:

- in the network trace from a web browser
- in log files for other services

An example of this type of problem is a job submitted by a SAS solution. As the job runs, it encounters an error and fails. The logs for the job might show that error was related to a call to another service. To troubleshoot these problems, you must get the log files for the servers and services associated with the affected request. For tools that you can use, see [“Ways to View Log Messages” in *SAS Viya Platform: Log Monitoring*](#).

To start investigating these symptoms and narrow the source of the problem, see [“Verify That Pods Are Ready” on page 5](#).

For issues with SAS Compute server, SAS Compute service, SAS Launcher service, or SAS Workload Orchestrator, see [“Troubleshoot a Failure to Start Compute Server” on page 21](#).

Infrastructure

Infrastructure refers to objects outside of SAS Viya platform that SAS Viya platform relies on to operate. Infrastructure can include these items:

- the underlying nodes
- the Kubernetes processes
- Ingress

Typically, problems in the infrastructure can appear in the following ways:

- as jobs that cannot start
- as services that cannot be reached

To start investigating these symptoms and narrow the source of the problem, see [“Verify That Pods Are Ready” on page 5](#).

First Steps

Verify That Pods Are Ready

Run the Command

Verify that all SAS Viya platform pods are ready. The `sas-readiness` service (that is used in this command) checks the status of the SAS Viya platform to determine whether it is ready for use.

Use this command:

```
kubectl -n namespace get pods -l app=sas-readiness
```

where *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).

Review the Output

Here is an example of the command output:

NAME	READY	STATUS	RESTARTS	AGE
<code>sas-readiness-76d594f95c-cztfl</code>	1/1	Running	0	2d23h

When reviewing the command output, remember these key points:

- The Ready column shows two values for each row: x/y.
 - The x value (Running) indicates a pod is scheduled and containers are executing.
 - The y value (Ready) indicates that the pod and its containers can receive full service requests.
- When reviewed at start-up, the Ready column value of 1/1 indicates that all of the pods are ready. However, after SAS Viya platform is running and performing operations, it is possible for SAS Viya platform pods to go offline to the point where SAS Viya platform is not usable. Despite this situation, the `sas-readiness` pod might continue to report 1/1 if Kubernetes-level readiness checks are still passing.
- The Running status indicates only that the `sas-readiness` pod is running and checking to determine whether other pods are ready.

If the `sas-readiness` pod shows a state of 1/1, all SAS pods are ready.

If the state is not 1/1, at least one pod is not in a ready state. The next step is to identify which pod (or pods) is not in a ready state.

Examine the sas-readiness Log

You can view messages in the `sas-readiness` log for the readiness pod. See [“Review the Logs” in SAS Viya Platform: Deployment Guide](#) for the command.

It is possible a single faulty pod or a small group of faulty pods is identified in the log. If a faulty pod has one or more services that are checked by the start-up sequencer, then the readiness pod log also lists all of the services that are waiting to start after the faulty pod.

To identify the specific pods that are faulty, review [“Look for Symptoms” on page 8](#). Then, go to [“Identify the Pods That Are Not Running” on page 9](#).

In this example output, the first two checks shown are for the `sas-database` and the `sas-oauth-provider`. Both have failed. The third check the 166 services (not shown for brevity) that failed because they were waiting on the database and `sas-oauth-provider-ready`.

Note: When SAS Viya platform is deploying or starting up, these errors do not necessarily indicate an issue. Monitor the `sas-readiness` log to ensure that these failure messages progressively disappear. However, if the messages remain, then review the logs for other pods.

```
INFO 2024-06-09 21:04:25.096 +0000 [sas-readiness] - The check "sas-database-ready" failed -
error retrieving application config from consul:
Get "https://sas-consul-server:8500/v1/kv/config/readiness?recurse=":
dial tcp 192.168.21.134:8500: connect: connection refused
INFO 2024-06-09 21:04:25.096 +0000 [sas-readiness] - The check "sas-oauth-provider-ready" failed -
No sas-logon-app pod is available yet.
INFO 2024-06-09 21:04:25.096 +0000 [sas-readiness] - The check "sas-endpoints-ready" failed -
166 endpoints have no available addresses:
```

If the initial investigation detects one or more pods are not running, see [“Troubleshoot Pod Failures” on page 8](#).

See Also

[“Readiness Service” in SAS Viya Platform: Deployment Guide](#)

Determine When the Problem Occurred

When you know the exact time of the problem (YYYY-MM-DD, hh:mm:ss), you can focus on the event log messages and metrics during a specified period of time.

IMPORTANT When using tools that display log messages, it can be unclear whether the time shown by the tool is GMT or your local time zone. Be sure to determine the time zone used.

See Also

- [“View Log Messages since a Specified Date” in SAS Viya Platform: Log Monitoring](#)
- [“Performing Basic Tasks in OpenSearch Dashboards” in SAS Viya Platform: Using SAS Viya Monitoring for Kubernetes](#)

Check Kubernetes Events

Check Kubernetes events for failures. Use this command:

```
kubectl -n namespace get events
```

Where *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).

IMPORTANT Be sure to check both of these namespaces:

- your SAS Viya platform namespace
- the kube-system namespace

Review Metric Data

Review metric-monitoring data for sizing issues such as pods that have reached their limit for memory or CPU.

Troubleshoot Pod Failures

Before You Start Troubleshooting

- 1 See [“Verify That Pods Are Ready” on page 5](#).
- 2 If the problem occurs during deployment or start up, consider waiting. SAS Viya platform can present failures that are actually temporary. Here are examples:
 - Kubernetes Cluster Autoscaler requires several minutes to spin up a new node.
 - Kubernetes requires several minutes to pull down large containers.In many cases, SAS Viya platform resolves these temporary failures as time passes.
- 3 Install a tool to improve the readability of JSON output. See [“About Kubectl Command Output” on page 60](#).

Look for Symptoms

Here are symptoms and situations that you should identify if they are present. They can narrow your troubleshooting path.

- What is the role of the affected pod. If more than one pod is involved, are the pods' purposes linked in some way?

For example, is it a stateful pod such as sas-consul-server (SAS Configuration Server) or sas-message-broker-server (SAS Message Broker), a SAS solution-specific pod, a SAS Compute or SAS CAS pod?
- How many pods are failing?
- How frequently is the pod restarting?
- Is there an out-of-memory kill (OOMKill) situation?
- What are the observed utilization metrics during the restart phase of the service? For example, review CPU utilization and RAM usage statistics (total RAM, used RAM, and available RAM).
- Are there any changes in the environment during this time period? Here are some examples:
 - ☐ updates to the system
 - ☐ changes in the workload such as a bulk data load, batch jobs, or users added
 - ☐ updates to the Kubernetes infrastructure

Identify the Pods That Are Not Running

Understand the Sequence of Pod Start Up

Before troubleshooting which pods are not running, it is helpful to know the expected order of pod start-up.

Note: Pod names might change between releases of SAS Viya platform. See [“Pod Changes by Release” in SAS Viya Platform Operations: Updating Software Tasks](#).

- 1 stateful pods
 - a sas-consul-server
 - b Postgres (and the data server operator)
 - c sas-rabbitmq-server
 - d sas-arke
 - e sas-redis-server
- 2 stateless pods
 - a core authentication- and authorization-related pods (sas-logon-app, sas-authorization, sas-configuration, sas-identities)
 - b other platform pods (sas-credentials, sas-files, sas-folders, sas-types, sas-cas-operator, and so on)
 - c all remaining services and web applications

Check the Pods in Order of Starting Up

Typically, the start-up sequencer is deployed and enabled by default. When it is, pods start in the order described in [“Understand the Sequence of Pod Start Up” on page 9](#).

Note: sas-startup-sequencer keeps some pods in the init state until other pods enter a ready state. For details, see [“Optimize Service Startup with SAS Startup Sequencer” in SAS Viya Platform: General Management of Servers and Services](#).

Check the pods that sas-startup-sequencer starts before other dependent pods. Use this command:

```
kubectll -n namespace get pods -l 'app in(sas-consul-server,sas-data-server-operator,sas-rabbitmq-server,sas-arke,sas-redis-server,sas-logon-app,sas-identities,sas-configuration,sas-authorization,sas-job-execution,sas-folders) '
```

where *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).

This example output indicates that all of the applicable pods are ready:

NAME	READY	STATUS	RESTARTS	AGE
sas-arke-85b679b86-qp4lt	1/1	Running	0	4d22h
sas-authorization-55864f779d-h6rvs	1/1	Running	0	4d22h
sas-configuration-7bc7f5868f-tv8kj	1/1	Running	0	4d22h
sas-consul-server-0	1/1	Running	0	4d22h
sas-consul-server-1	1/1	Running	0	4d22h

Note: In this output, “ready” indicates that Kubernetes sees the container running normally. This is a different indicator than the `READY` column seen in the `sas-readiness` log output.

Determine the Order If Multiple Pods Fail to Start

When a pod fails to start, it can be useful to check its `sas-start-sequencer` container. If the pod has a dependency on another pod or service, the `sas-start-sequencer` log indicates whether the failing pod is waiting on that other pod or service to become available. Use this command:

```
kubectl -n namespace logs pod_name -c sas-start-sequencer
```

Where:

- *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).
- *pod_name* is the pod that you want the command to affect.

Identify Pods That Are Not Ready

Check `sas-consul-server`. Use this command:

```
kubectl -n namespace get pods -l app=sas-consul-server
```

where *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).

Consider identifying any pods that are not in a good state. Use this command:

```
kubectl -n namespace get pods -o jsonpath='{range .items[?(@.status.containerStatuses[].ready==false)]}{.metadata.name}{ " "}{.status.phase}{ "\n"}' |
grep -v Succeeded
```

Identify the pods on which many other pods depend. Use this command:

```
kubectl -n namespace get pods -l 'app in(sas-consul-server,sas-data-server-operator,sas-rabbitmq-server,sas-arke,sas-redis-server,sas-logon-app,sas-identities,sas-configuration,sas-authorization,sas-job-execution,sas-folders) '
```

Note:

- When reviewing the pods that are not ready, examine them in the order described in [“Understand the Sequence of Pod Start Up” on page 9](#). For example, if both sas-consul-server and sas-logon-app are not ready, examine sas-consul-server first because it is earlier in the sequence.
- Possible pod states are Pending, Running, and Succeeded.

Here is example output:

```
sas-consul-server-0 Running
sas-consul-server-1 Running
sas-consul-server-2 Running
sas-credentials-6df6f557c9-hxprh Pending
sas-crossdomainproxy-9d45bd985-nk7jx Pending
```

You can retrieve the same output with more information when you use `kubectl get pods`:

NAME	READY	STATUS	RESTARTS	AGE
sas-consul-server-0	1/1	Running	0	79s
sas-consul-server-1	1/1	Running	0	79s
sas-consul-server-2	1/1	Running	0	79s
sas-credentials-6df6f557c9-hxprh	0/1	Init:0/2	0	103s
sas-crossdomainproxy-9d45bd985-nk7jx	0/1	Init:0/2	0	103s

Determine whether data-server-operator is running. It is useful to check it before sas-logon-app and sas-identities. Use this command:

```
kubectl -n namespace get pods -l app=sas-data-server-operator
```

To check the logs for data-server-operator, use this command:

```
kubectl -n namespace logs -l app=sas-data-server-operator | grep Reconcile
```

If your deployment uses an internal PostgreSQL database, examine the state of its pods. Use this command:

```
kubectl -n namespace get pods -l postgres-operator.crunchydata.com/cluster=sas-crunchy-
platform-postgres
```

This example output (shortened for brevity) indicates that all of the applicable pods are ready:

NAME	READY	STATUS	RESTARTS	AGE
sas-crunchy-platform-postgres-00-4rww-0	5/5	Running	0	4d22h
sas-crunchy-platform-postgres-00-s85p-0	5/5	Running	0	4d22h
sas-crunchy-platform-postgres-00-v7br-0	5/5	Running	0	4d22h
sas-crunchy-platform-postgres-repo-host-0	2/2	Running	0	4d22h

Identify any remaining pods that are not ready. Use this command:

```
kubectl -n namespace get pods -o jsonpath='{range .items[?
(@.status.containerStatuses[].ready==false)]}{.metadata.name}{ " "}{.status.phase}{ "\n"}
```

Note: The space between “range” and “.items” is required.

Identify Pods with Excessive Restarts

Kubernetes attempts to restart failed containers in a pod automatically. However, frequent restarts can indicate problems such as these examples:

- A pod is running low on resources (such as memory).
- A pod is unable to establish a connection to another pod.

Look for any pods with an excessive number of restarts, typically five or more. Use this command:

```
kubectl -n namespace get pods -o jsonpath='{range .items[?(@.status.containerStatuses[].restartCount>5)]}{.metadata.name}{ " "}{.status.phase}{ "\n"}'
```

where *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).

Note: The space between “range” and “.items” is required.

Review Events Associated with the Entire Namespace

Review the events for the entire namespace to see whether there are any failure or warning messages related to these symptoms:

- pod termination
- inability to schedule pods

Note: By default, Kubernetes retains these events for the last hour only. However, the retention value is a configurable parameter. See `--event-ttl` at [Kubernetes: kube-apiserver](#)

Use this command, which returns the events sorted by their timestamps:

```
kubectl -n namespace get events --sort-by='.lastTimestamp'
```

where *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).

The output from this command provides an overview of the environment. Then, you can filter the output to show only non-normal events. Use this command:

```
kubectl -n namespace get events --field-selector type!=Normal
```

See Also

[Kubernetes: Pod Lifecycle](#)

Investigate Why the Pod Is Not Running

After you identify the service pod that is not running, use these steps to investigate the service.

Learn More about the Faulty Pod

View the describe output for the affected pod to learn more about it. The output includes Kubernetes events that are specific to the pod. Use this command:

Note: By default, Kubernetes retains these events for the last hour only. However, the retention value is a configurable parameter. See `--event-ttl` at [Kubernetes: kube-apiserver](#)

```
kubectl -n namespace describe pod pod_name
```

Where:

- *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).
- *pod_name* is the pod that you want the command to affect.

By using this command, you can check the following information:

- the events associated with the pod
- the status of each container within the pod by looking at the **State**, **Ready**, and **Restart Count** fields.

This example command shows how to learn more about the sas-microanalytic-score service in the viya namespace:

```
kubectl -n viya describe pod sas-microanalytic-score-767d589599-t85fw | grep -E '^Events' -A20
```

The `grep` command locates the Events section in the describe command output. The “Events” heading is extracted and `-A20` indicates to extract the 20 lines (or fewer) that follow the “Events” heading. You can specify a number other than 20.

TIP Consider sending the output to a file.

The following example output indicates no problems because the event Type is Normal:

```
Events:
  Type    Reason      Age   From          Message
  ----    -
  Normal  Scheduled   10m   default-scheduler  Successfully assigned viya/sas-compute-server-20983df4-4bf5-4b32-b57a-760a2dba3f8e-43945 to aks-compute-21054622-vmss0000ft
  Normal  Pulled      10m   kubelet        Container image "cr.sas.com/viya-4-x64_oci_linux_2-docker/sas-certframe:3.53.0-20240502.1714673208358" already present on machine
  Normal  Created     10m   kubelet        Created container sas-certframe
  Normal  Started     10m   kubelet        Started container sas-certframe
  Normal  Pulled      10m   kubelet        Container image "cr.sas.com/viya-4-x64_oci_linux_2-docker/sas-config-init:1.22.4-20240403.1712157445848" already present on machine
  Normal  Created     10m   kubelet        Created container sas-config-init
  Normal  Started     10m   kubelet        Started container sas-config-init
  Normal  Pulled      10m   kubelet        Container image "cr.sas.com/viya-4-x64_oci_linux_2-
```

				docker/sas-programming-environment:1.56.22-20240514.1715682514463" already present on machine
Normal	Created	10m	kubelet	Created container sas-programming-environment
Normal	Started	10m	kubelet	Started container sas-programming-environment
Normal	Pulled	10m	kubelet	Container image "cr.sas.com/viya-4-x64_oci_linux_2-docker/sas-config-init:1.22.4-20240403.1712157445848" already present on machine
Normal	Created	10m	kubelet	Created container refreshtoken
Normal	Started	10m	kubelet	Started container refreshtoken
Normal	Pulled	10m	kubelet	Container image "cr.sas.com/viya-4-x64_oci_linux_2-docker/sas-process-exporter:1.10.4-20240501.1714568043041" already present on machine
Normal	Created	10m	kubelet	Created container sas-process-exporter
Normal	Started	10m	kubelet	Started container sas-process-exporter

If there are no event messages that indicate issues that might prevent the pod from starting, then go to [“Review the Log” on page 15](#).

Check the Init Container

Most pods have a set of init containers (such as sas-config-init, sas-db-migrator, sas-hydrator, sas-certframe, and sas-start-sequencer) in addition to the main “service” container (such as sas-authorization, sas-logon-app, and sas-types). Check to see whether each init container has completed and does not have a failed status. The output from the following command includes state of the init containers as well as the regular containers:

```
kubectl -n namespace describe pod pod_name
```

Where:

- *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).
- *pod_name* is the pod that you want the command to affect.

The following example output (shortened for brevity) shows the status of the sas-start-sequencer init container:

```
Init Containers:
  sas-start-sequencer:
  ...
    State:          Terminated
      Reason:       Completed
    Exit Code:      0
    Started:        Sun, 07 Jul 2024 22:36:08 -0400
    Finished:       Sun, 07 Jul 2024 22:36:11 -0400
    Ready:          True
    Restart Count:  0
```

TIP You can retrieve comparable information by using `kubectl get pod`. Its **Ready** column indicates “Init” x/x. The value is based on the number of init containers (out of the total) that have completed.

If the status of the init container is failed, review the log for the affected init container. Use this command:

```
kubectll -n namespace logs pod_name -c init_container_name
```

If the status of the init container has not failed, view the status of the other containers and review their logs. See [“Review the Log” on page 15](#).

Review the Log

Review the log messages for the affected service pod to determine why it is not ready.

Note: It is possible that a log message is labeled an error but it is not, in fact, an issue of concern.

Use this command:

```
kubectll -n namespace logs pod_name
```

Where:

- *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).
- *pod_name* is the pod that you want the command to affect.

To further refine the log message information, see these topics:

- [“Display the Log Messages for a Specific Container within a Pod” in SAS Viya Platform: Log Monitoring.](#)
- [“View Log Messages for a Specific Amount of Time” in SAS Viya Platform: Log Monitoring.](#)
- [“View a Specified Number of the Most Recent Log Messages” in SAS Viya Platform: Log Monitoring.](#)
- [“View Log Messages since a Specified Date” in SAS Viya Platform: Log Monitoring.](#)

Here are other ways to view log message information:

- Use OpenSearch Dashboards. See [“Performing Basic Tasks in OpenSearch Dashboards” in SAS Viya Platform: Using SAS Viya Monitoring for Kubernetes.](#)
- Use the getlogs.py script. See [“Exporting Logs with getlogs.py” in SAS Viya Platform: Using SAS Viya Monitoring for Kubernetes.](#)

Review Prior Logs

View prior instances of the log if they exist. Prior logs can indicate why the pod restarted. This information can potentially diagnose an out-of-memory kill (OOMKill) situation.

```
kubectll -n namespace logs pod_name --previous
```

Where:

- *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).

- `pod_name` is the pod that you want the command to affect.

Resolve the Problem

TIP Use the Perf / Analysis Grafana dashboard to visualize memory and CPU usage of pods. Prometheus and Grafana open-source tools are included in the SAS Viya Monitoring for Kubernetes project. See [“Dashboards for Kubernetes Resources” in SAS Viya Platform: Deploying SAS Viya Monitoring for Kubernetes](#).

Restart or Delete the Pod

When you restart or delete the pod, a new pod is created, which might fix the error. If the error is not fixed, this still provides these advantages:

- Events that might have timed out are repeated.
- The new log is cleaner and easier to review.

Note: When troubleshooting, remember these key points:

- When you delete a SAS Compute server pod, you might need to resubmit any job associated with that pod.
- Deleting the CAS server pods stops the running CAS servers, drops all the in-memory data, and restarts the new pods. This causes an outage that is noticeable for the end users.

There are two ways to delete a pod: a soft or hard deletion.

A soft deletion via a rollout restart

A soft deletion enables you to correct the problem pod with no service outage and thus less disruption. The `rollout restart` works in the following ways:

- **For DaemonSets and Deployment object types:** The `rollout restart` starts a new instance of each pod in DaemonSets and Deployment object types. Then, the command removes the old pod instance after the new pod is in a Ready state.
- **For StatefulSets:** The `rollout restart` stops the old pod instance. Then, the command starts a new pod instance, one at a time.

Note: In StatefulSets the pod name is static (such as, `sas-rabbitmq-server-0`, `sas-rabbitmq-server-1`, `sas-rabbitmq-server-2`).

Use this command:

```
kubectl -n namespace rollout restart object_type/short_pod_name
```

Where:

- *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).
- *object_type* is the type of object that you want the command to affect. In this context, valid values are Deployment, StatefulSet, or DaemonSet.
- *short_pod_name* is the pod that you want the command to affect (for example, sas-folders, not sas-folders-7f49c8b54d-p5vd8).

The following are examples of this command:

```
kubectl rollout restart deploy/sas-authorization
kubectl rollout restart sts/sas-workload-orchestrator
```

A hard deletion via a delete

Soft deletion does not always work. Then, you have two pod instances in a state that is not Running and Ready.

A hard deletion terminates the specified pod. This triggers Kubernetes to start a new pod if it was associated with a resource object (Deployment, DaemonSet, or StatefulSet). A hard deletion can disrupt users while the replacement pod starts up.

Use this command:

```
kubectl -n namespace delete pod pod_name
```

Increase the Memory Limit for a Pod

A pod might require that you increase its memory limit. First, you must identify the current memory limit for the affected pod. Depending on the resource, the pod might be managed by a Deployment, DaemonSet or StatefulSet object type.

Use this command:

```
kubectl -n namespace get object_type resource_name -o
jsonpath='{.spec.template.spec.containers[container_index].resources.limits.memory}'{"\n"}
```

Where:

- *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).
- *object_type* is the type of object that you want the command to affect. In this context, valid values are Deployment, StatefulSet, or DaemonSet.
- *resource_name* is typically the prefix for the pod or pods that you want the command to affect (for example, sas-folders, not sas-folders-7f49c8b54d-p5vd8 [Deployment], or sas-rabbitmq-server instead of sas-rabbitmq-server-0 [StatefulSet], or sas-workload-orchestrator instead of sas-workload-orchestrator-lww5l [DaemonSet]).
- *container_index* is the index to the affected container. Most pods have one container and the index is 0 (zero). For pods with multiple containers, you must identify the container on which to adjust the memory limits. Then, specify that index when updating the value.

To modify this value without redeploying or updating the release of SAS Viya platform, use this command:

```
kubectl -n namespace patch object_type resource_name --type json -p='[{"op":"replace",
"path":"/spec/template/spec/containers/container_index/resources/limits/
memory","value":"new_memory_limit"}]'
```

where *new_memory_limit* is the increased memory limit for the affected pod. This value must be in Kubernetes units. For details, see [Kubernetes: Memory Resource Units](#).

If you want to persist the memory-limit change, you must create and add a PatchTransformer section to the `kustomization.yaml` file.

IMPORTANT The PatchTransformer option can encounter difficulties when updating from one SAS Viya platform release to another. Here are some examples:

- If the pod is merged with other pods and given a new name in the newer release, the transformer cannot find any matches and fails.
- If, for some reason, the affected pod receives an additional container in the newer release, the reference to the *container_index* in the path might subsequently point to the wrong location.

Use this syntax:

```
apiVersion: builtin
kind: PatchTransformer
metadata:
  name: unique_patch_name
patch: |-
  - op: replace
    path: /spec/template/spec/containers/container_index/resources/limits/memory
    value: new_memory_limit
target:
  kind: object_type
  name: resource_name
```

where *unique_patch_name* is a unique name for the patch.

Note: If the affected pod is created by SAS Launcher service, the memory-limit value is managed by the `launcher.default` and `launcher.max` configuration settings. As of 2024.06, you can manage this setting at the SAS Launcher context level. See [Manage Requests and Limits for CPU and Memory](#) and “[Configuration Properties: Launcher Service](#)” in *SAS Viya Platform: Programming Run-Time Servers*.

Here is an example of a PatchTransformer section:

```
apiVersion: builtin
kind: PatchTransformer
metadata:
  name: patch-identities-mem
patch: |-
  - op: replace
    path: /spec/template/spec/containers/0/resources/limits/memory
    value: 2Gi
target:
  kind: Deployment
```

name: sas-identities

Adjust Configuration Memory Settings

If you have pods that are being killed for using too much memory (OOMKilled status), the following memory-limit settings are a potential cause:

Java -Xmx setting

Some of the pods are running Java services. Java services have a memory heap size limit setting called `-Xmx`. You can configure it in the JVM definition for the affected service by using SAS Environment Manager.

IMPORTANT If you increase this setting from the default value, it is also important to increase the Kubernetes memory limit for the pod. If you do not make this adjustment, Kubernetes kills the pod because the pod attempts to use more than its allowed memory.

SAS Viya platform MEMSIZE setting

SAS Viya platform has a similar setting called `MEMSIZE`. When you increase this value, you must make a corresponding increase in the SAS Viya platform configuration (`sas.launcher.default` and `sas.launcher.max`).

See [“Edit Configuration Instances” in SAS Environment Manager: User’s Guide](#). Also, see [SAS Viya Platform: Configuration Properties](#).

An overcommitted node can also cause too much memory to be used. Here is an example:

- 1 You have 100 pods. Each has requested 50 MiB memory, but 2 GiB is their upper limit.
- 2 All of the pods attempt to use 2 GiB.
- 3 The node has only 64 GiB (not 200+ GiB).
- 4 The node experiences memory pressure and starts killing pods to resolve the problem.

To fix this problem, you must ensure that your node size and quantity match your workload. Consider tuning your requests (instead of only the limits) to more closely match what you expect the workload to require.

See Also

- [“Tuning Workload Allocations” in SAS Viya Platform Administration: Tuning](#)
- [“Plan the Workload Placement” in SAS Viya Platform: Deployment Guide](#)
- [“Manage Requests and Limits for CPU and Memory” in SAS Viya Platform: Programming Run-Time Servers](#)
- [“Configuration Properties: Launcher Service” in SAS Viya Platform: Programming Run-Time Servers](#)

Configure High Availability (HA)

High Availability (HA) is an optional configuration that updates the Horizontal Pod Autoscaler (HPA) configuration with a minimum of two replicas for each service. It also sets the Pod Disruption Budget (PDB) for those services to a minimum available of one. This helps to ensure that the service remains available in either of the following situations:

- One of the pods handling that service fails.
- A Kubernetes event triggers the service to terminate and move to another node.

Because High Availability runs multiple instances of each service, the cluster needs more capacity to accommodate these instances than when HA is not configured.

If service availability issues are the result of pods being frequently moved by Kubernetes, this might suggest the underlying Kubernetes infrastructure is unstable or undersized for the existing workload. If so, address these issues before implementing high availability.

See Also

[“Configure High Availability” in SAS Viya Platform: Deployment Guide](#)

Configure Horizontal Pod Autoscaler (HPA)

You can configure Kubernetes to scale services based on utilization via Horizontal Pod Autoscaler (HPA). Doing so adds more pods for a given service if the service becomes too busy.

IMPORTANT This feature does not apply to all of the SAS Viya platform. For example, CAS servers do not auto-scale.

See Also

[“Configure High Availability” in SAS Viya Platform: Deployment Guide](#)

Use Taints and Labels

Taints and labels can prevent pods from running on nodes which they should not. When pods do run on inappropriate nodes, it can take resources from service pods and impact the environment as a whole.

See Also

- [“Tuning Workload Allocations” in SAS Viya Platform Administration: Tuning](#)
- [“Plan the Workload Placement” in SAS Viya Platform: Deployment Guide](#)

Modify Configuration Properties of a Service

When a service fails to start, it might be one of these problems:

- A problem with the configuration of the affected service.
- An inability to perform some action that the affected service requires to start up (such as bootstrapping).

See Also

- [“Configuration Properties by Component” in SAS Viya: Configuration Dictionaries](#)
- [“Edit Configuration Instances” in SAS Environment Manager: User’s Guide](#)

Troubleshoot a Failure to Start Compute Server

Start Here

IMPORTANT Before you start, be sure you understand the following information:

- [“First Steps” on page 5](#)
- [“How Compute Server Works” in SAS Viya Platform: Programming Run-Time Servers](#)
- [“Hosts and Pods” in SAS Viya Platform: Workload Management](#)

Compute server can fail due to many reasons. The process to create a Compute server involves several components:

- SAS Compute server.
- SAS Compute service. It can also act as SAS Compute provider.
- SAS Launcher service.
- SAS Workload Orchestrator.
- SAS Infrastructure Data Server (Platform PostgreSQL)

Any of these components might be the cause of the failure. Therefore, you can start troubleshooting in any of the following ways:

- [“Determine How the Compute Server Pod Was Created” on page 22](#)

- “Examine the Compute Service Logs” on page 24
- “Examine the Launcher Service Logs” on page 27
- “Avoid Database Issues” on page 55

Information that you learn from these topics can help you identify how to further troubleshoot the failure:

- “Investigate a Failure Related to SAS Workload Orchestrator” on page 28
- “Investigate a Failure Related to SAS Launcher Service” on page 32

TIP

- This topic provides kubectl commands for troubleshooting. An alternative is to use a monitoring tool. If you have SAS Viya Monitoring for Kubernetes deployed, you can use OpenSearch Dashboards to view log messages over a range of time. See [“Examining Log Messages with OpenSearch Dashboards”](#) in *SAS Viya Platform: Using SAS Viya Monitoring for Kubernetes*.
- You can also monitor metric data by using Grafana dashboards. Grafana is also provided by SAS Viya Monitoring for Kubernetes. See [“Monitoring Jobs Started by SAS Launcher Service”](#) in *SAS Viya Platform: Using SAS Viya Monitoring for Kubernetes*.

Determine How the Compute Server Pod Was Created

Decipher the Compute Server Pod Name

The Compute server pod name indicates how it was created and other useful information. A SAS Compute server pod can be created by either SAS Launcher service or SAS Workload Orchestrator. To troubleshoot effectively, you must determine which service created the failed Compute server pod.

IMPORTANT Be sure you understand [“How Compute Server Works”](#) in *SAS Viya Platform: Programming Run-Time Servers*.

When reviewing log messages and other output for Compute server, Compute service, Launcher service, or SAS Workload Orchestrator, remember these key points:

- The “Kubernetes resource name” indicates the pod name of the Compute server that is in use.
- By default, the Compute server pod name uses the prefix `sas-compute-server-`.

This default can be overridden by the following items:

- Launcher context

Check the context definition to verify the prefix in use for Launcher service. See [“job.name.prefix” in SAS Viya Platform: Programming Run-Time Servers](#).

- ❑ Compute context

Check the compute session attribute to verify the prefix in use for Compute server. See [“Compute Session Attributes” in SAS Viya Platform: Programming Run-Time Servers](#).

- ❑ Job Execution Service requests

When Job Execution Service (JES) requests that a Compute server start in order to execute a job, the Compute server pod name prefix is changed to the name of the job. For details, see [“About Compute Server and Job Execution Service Job Names” on page 60](#).

For concepts, see [“How Jobs and Flows Work” in SAS Viya Platform: Jobs and Flows](#).

Note: If a reusable Compute server is in use, the Compute server pod is already named. See [Configure Reusable Compute Servers](#) and [“Reusable Compute Server” in SAS Viya Platform: Programming Run-Time Servers](#).

- The Launcher process ID (for the SAS Launcher service) is appended to the Compute server prefix:

```
"sas-compute-server-144bfea2-dd39-4499-90c5-bbeeaf2f4b6f"
```

This string represents the Kubernetes job object that is created by the Launcher service.

- When the Launcher process ID is followed by a hash, the string represents a Kubernetes pod:

```
"sas-compute-server-144bfea2-dd39-4499-90c5-bbeeaf2f4b6f-9chqw"
```

The Kubernetes job object (that was created by the Launcher service) creates this Kubernetes pod.

- When the Launcher process ID is followed by a numeral instead of a hash, that numeral is the SAS Workload Orchestrator job ID:

```
"sas-compute-server-144bfea2-dd39-4499-90c5-bbeeaf2f4b6f-243"
```

This string represents the Kubernetes pod created by SAS Workload Orchestrator.

- SAS Workload Orchestrator server (DaemonSet) creates pods with names that end in a hash:

```
sas-workload-orchestrator-cvb26
```

- SAS Workload Orchestrator manager (StatefulSet) creates pods with names that end in a single digit:

```
sas-workload-orchestrator-0
```

Identify the Name of the Compute Server Pod That Failed to Start

When Compute server fails to start, there might not be a pod from which to get a log. Instead, you can review Kubernetes events. These events might show errors that occurred during the creation of the pod's containers. Also, these errors might contain the Compute server pod name, which you can use to learn more about the issue that caused the failure. To view Kubernetes events, use this command:

```
kubectl -n namespace get events
```

where *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).

TIP This command can generate a long list. However, do not narrow the list immediately. The issue might have occurred at the cluster level, and therefore no pod has been scheduled. The complete list can help you begin to understand the issue.

In the output, look for the label “Kubernetes resource name”.

- This example indicates that the Launcher service created the Kubernetes pod.

Kubernetes resource name: "sas-compute-server-144bfea2-dd39-4499-90c5-bbeeaf2f4b6f".

To troubleshoot this path, go to [“Investigate a Failure Related to SAS Launcher Service” on page 32](#).

- This example indicates that SAS Workload Orchestrator created the Kubernetes pod.

Kubernetes resource name: "sas-compute-server-144bfea2-dd39-4499-90c5-bbeeaf2f4b6f-243".

To troubleshoot this path, go to [“Investigate a Failure Related to SAS Workload Orchestrator” on page 28](#).

Examine the Compute Service Logs

IMPORTANT Be sure you understand [“How Compute Server Works” in SAS Viya Platform: Programming Run-Time Servers](#). This topic describes the relationship among *compute service*, *compute session*, and *compute server*.

The Compute service logs can provide useful information when troubleshooting Compute server.

- 1 List the Compute service pods. Use this command:

```
kubect1 -n namespace get pods -l app=sas-compute
```

where *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).

- 2 From the list, identify the Compute service pod for which you want to view the log messages. Depending on your deployment, there might be only one Compute service pod or many. When there are many pods, review the log messages for each pod in order to identify the pod to investigate further. Use this command:

```
kubect1 -n namespace logs pod_name
```

where *pod_name* is the Compute service pod whose logs you want to view.

TIP To identify the affected Compute service pod, use these methods:

- If one Compute server failed, scan each Compute service pod's logs for any error messages. The log with error messages identifies the affected Compute server pod.
- If a Compute server pod did not reach a Running state before the time-out, there should be a message similar to this example:

```
RequestID requestID: The launcher process did not reach a running state
before the seconds second timeout was reached.
```

Where:

- *requestID* is the request ID is associated with the Compute session.
- *seconds* is the configured Compute server launch time-out period. The default is 60 seconds.
- If you know the Compute server pod's name or the Launcher process ID, you can search for that value in the logs also.

- 3 After you identify the affected Compute service pod, look for a message in its log that is similar to this example:

```
RequestID 9e669ce1-05f3-44ad-9da4-d2b4c48e6500:
```

```
A session has been requested for the context with the ID "47fba9da-7f66-42d1-93f9-6e068d60a703"
```

This message occurs when the Compute service receives a request to start a new Compute session. Notice that a request ID is provided. Every log message in the Compute service logs that is related to the processing of that request is prefixed with that request ID. When troubleshooting Compute service, make a note of the request ID associated with the Compute session.

TIP Compute service can process multiple requests at any time. To identify a specific request ID, use these methods:

- Narrow the requests by time. Identify a range of time when the failure occurred. Then examine the request IDs with timestamps during that time range. See [“Determine When the Problem Occurred” on page 7](#).
- Narrow the requests by context ID. Identify the context ID passed to Compute service to use in the request. Then match that context ID to the context ID in the session request message.
- Review the log messages for error messages. Identify the request ID associated with those messages. If only one request ID is associated with all of the error messages, then that is the affected request.

- 4 Compute service performs internal processing before calling the Launcher service.

TIP If something fails during this processing, an HTTP response from the REST API might contain helpful information. Typically, this information appears as an error in the Compute service log, the log of the client requesting the Compute server (such as SAS Studio), or both.

If you request a Compute server by using the REST API itself, the REST API response contains an error when there is a failure.

- 5 When processing completes successfully, Compute service calls the Launcher service to create a new Compute server.
- 6 After Launcher service responds to Compute service, Compute service logs a message similar to this example:

```
RequestID 9e669ce1-05f3-44ad-9da4-d2b4c48e6500:
The server with the ID "81b8b29a-f351-4835-9860-bda782faafe7"
is associated with the following resources:
Compute context ID "47fba9da-7f66-42d1-93f9-6e068d60a703";
Launcher process ID: "3fb669bf-dea6-4c8a-ae31-63426f775601";
Kubernetes resource name: "sas-compute-server-3fb669bf-dea6-4c8a-ae31-63426f775601-280"
```

Notice that the request ID matches the example shown in step [Step 3 on page 25](#).

This message enables you to tie the session request ID to its associated server ID, Launcher process ID, and the Kubernetes resource name (which is the Compute server pod name). These additional IDs enable you to identify and then view the following information sources:

- SAS Launcher service logs by using the Launcher process ID. See [“Examine the Launcher Service Logs” on page 27](#).
- SAS Workload Orchestrator logs by using the Kubernetes resource name and the SAS Workload Orchestrator job ID (280 in this example message). See [“Investigate a Failure Related to SAS Workload Orchestrator” on page 28](#).
- Kubernetes event messages by using the Launcher process ID or the Kubernetes resource name depending on the resource that you are investigating.

Note: The server ID is used exclusively within the Compute service. It enables you to associate the messages for the Launcher process ID and the Kubernetes resource name with each other. The server ID is unknown outside the Compute service.

Inside the Compute service, the server ID is used in this way:

- **(For non-reusable servers)** The server ID is linked to an individual session ID because that server runs only one session.
 - **(For reusable servers)** The server ID is linked to multiple session IDs because that server can run multiple sessions over its lifetime. See [Configure Reusable Compute Servers](#) and [“Reusable Compute Server” in SAS Viya Platform: Programming Run-Time Servers](#).
-

Note:

- If the Launcher service process does not achieve the Running state within the configured `LaunchTimeout` period (which defaults to 60 seconds), Compute service sends a request to the Launcher service to kill the process. No information is logged about the cause of the process

failing to reach the Running state. If this situation occurs, review the Kubernetes events associated with the Kubernetes resource name.

- The log message examples are INFO level and are present by default. The loggers do not require adjustment.
- If Launcher service sends events that contain Kubernetes errors in them, the Compute service logs contain these events. However, these are typically debug-level log messages. Also, these log messages do not contain a request ID. Instead, you must use the Kubernetes resource name to identify the messages associated with the Compute server that you are troubleshooting.

Examine the Launcher Service Logs

IMPORTANT Be sure you understand “[How Compute Server Works](#)” in *SAS Viya Platform: Programming Run-Time Servers*.

The Launcher service logs can provide useful information when troubleshooting Compute server.

- 1 View the log messages for all Launcher service pods. Use your preferred log-messages software or use this command:

```
kubect1 -n namespace logs -l app=sas-launcher
```

where *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see “[About the Current Namespace](#)” on page 60.

TIP This command shows only the last 10 lines of output from the SAS Launcher logs. To show more lines, add the `--tail` option to the command. To show 50 lines, specify `--tail=50`. To show all lines, specify `--tail=-1`.

- 2 Scan the log messages for any relevant errors. Specifically, look for any messages of these types:
 - Launcher service run-time errors
 - SAS Workload Orchestrator REST API calls.

TIP The Launcher service records into its log any relevant Kubernetes events. It is unnecessary to get the Kubernetes events separately.

- 3 If there are no error messages, look for a message similar to the following example:

```
Process ID "ab63d606-6438-4c3b-b613-8551ca1d19ba" was created by the user: "sasadm"
```

This message indicates one of these scenarios:

- A Kubernetes job object was created by Launcher service.
- A pod was created by SAS Workload Orchestrator.

- 4 List the Kubernetes pods that the Launcher service has created. Use this command:

```
kubectl -n namespace get pods -l sas.com/created-by=sas-launcher
```

The job names in the output indicate how Compute server was created. See [“Determine How the Compute Server Pod Was Created”](#) on page 22.

TIP An alternative is to verify that SAS Workload Orchestrator is enabled in your deployment. See [“Determine Whether SAS Workload Orchestrator Is Enabled”](#) in *SAS Viya Platform: Workload Management*.

If it is not enabled, then a Kubernetes job object was created by the Launcher service.

- 5 If the Compute server pod was created by SAS Workload Orchestrator, go to [“Investigate a Failure Related to SAS Workload Orchestrator”](#) on page 28.

Otherwise, go to [“Investigate a Failure Related to SAS Launcher Service”](#) on page 32.

Investigate a Failure Related to SAS Workload Orchestrator

Determine If SAS Workload Orchestrator Received a Request to Run a Compute Server

You can determine whether SAS Workload Orchestrator received any job submissions to start Compute server in the following ways:

- In SAS Environment Manager, review the **Jobs** table on the **Workload Orchestrator** page. See [“Managing Jobs”](#) in *SAS Environment Manager: User's Guide*.
- Use the SAS Workload Orchestrator plug-in to the sas-viya CLI.

```
sas-viya workload-orchestrator jobs list
```

See [“The Jobs Command”](#) in *SAS Viya Platform: Workload Management*.

If SAS Workload Orchestrator received a job submission with the same name as the Compute server, go to [“View the SAS Workload Orchestrator Job Status and Error Output”](#) on page 28.

If SAS Workload Orchestrator did not receive a job submission with the same name as the Compute server, then there are no SAS Workload Orchestrator diagnostics to help determine the problem. Instead, review the SAS Launcher service logs. See [“Examine the Launcher Service Logs”](#) on page 27.

View the SAS Workload Orchestrator Job Status and Error Output

Complete these steps:

- 1 Open the **Workload Orchestrator** page in SAS Environment Manager.
- 2 Select the **Jobs** tab.
- 3 Select the affected job in the table.
- 4 In the **Job details** page, view the jobs status. See the applicable topic:
 - [“Diagnose the LAUNCH_ERROR Job Status” on page 29](#)
 - [“Diagnose the KILLED Job Status” on page 31](#)
- 5 Select the **Output** tab. Any available errors are displayed on this tab.

Diagnose the LAUNCH_ERROR Job Status

The LAUNCH_ERROR job status indicates that SAS Workload Orchestrator failed to create the Compute pod. To view additional information about this status, see these sources of information:

- **Error output for the SAS Workload Orchestrator job.** Error output might be available. See step 5 in [“View the SAS Workload Orchestrator Job Status and Error Output” on page 28](#).
- **The SAS Workload Orchestrator DaemonSet log.** You can attempt to examine the log for the SAS Workload Orchestrator daemon pod that attempted the creation. The failures appear in its log. View log-monitoring data in your preferred tool or use this command:

```
kubectl -n namespace logs pod_name
```

Where:

- *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).
- *pod_name* is the pod that you want the command to affect.

Note: You can find the pod name for the SAS Workload Orchestrator daemon in the following ways:

- **(For SAS administrators)** The pod name is available from the job's execution host aliases. On the **Workload Orchestrator** page, in the **Jobs** tab, select the affected job. In the **Job details** page, view the **Host Information** tab.
- **(For Kubernetes administrators)** Attempt to match the following two nodes in which the Compute server pod is active:
 - The node on which the Compute server pod runs.
 - The node on which the SAS Workload Orchestrator DaemonSet pod runs. The SAS Workload Orchestrator DaemonSet pod manages the Compute server pod.

These nodes are the same. If you can match the nodes, then you can identify the SAS Workload Orchestrator DaemonSet pod name. However, this must be done before the Compute server pod ends.

-
- **In the Kubernetes events.** If you cannot get the logs, use the Kubernetes events. The events can contain logged errors that are generated during the creation of the containers. Use this command:

```
kubectl -n namespace get events
```

where *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).

When you review these sources of information, look for output that is similar to the following phrases.

TIP In this section, the problem might be due to an accidental deletion. You can remedy accidental deletions by redeploying the assets associated with the deleted item. Use the original deployment method (such as kubectl commands).

- **Container does not exist.** Output similar to this phrase can indicate the following issues and remedies:
 - The name of the container image is spelled incorrectly. Carefully review the spelling of the name and correct it.
 - The container image is no longer in the repository. Typically, this is due to a newer version replacing the container image. Identify and use the name of the newer container image.
- **Cannot pull container due to permissions problem.** Output similar to this phrase can indicate the following issues and remedies:
 - The image-pull secret specified in the pod template does not exist.
 - During an update to a newer release of SAS Viya platform, the secret was updated and the hash in its name changed, but the pod template was not updated to use the new name. Update the pod template to use the new name of the secret.
 - The secret name is misspelled. Carefully review the spelling of the name and correct it.
 - The secret was deleted accidentally.
 - The image-pull secret exists, but does not allow authentication.
Make sure that the image pull secret has correct information in it.
- **Cannot find configmap.** Output similar to this phrase can indicate the following issues and remedies:
 - During an update to a newer release of SAS Viya platform, the config map was updated and the hash in the name changed, but the pod template was not updated to use the new name. Update the pod template to use the new name of the config map.
 - The config map name is misspelled. Carefully review the spelling of the name and correct it.
 - The config map was deleted accidentally. The kubectl command might not have been applied completely (for example, it was applied to the namespace only). Rerun the commands to restore a consistent state.
- **Cannot find secret.** Output similar to this phrase can indicate the following issues and remedies:
 - During an update to a newer release of SAS Viya platform, the secret was updated and the hash in the name changed, but the pod template was not updated to use the new name. Update the pod template to use the new name of the secret.
 - The secret name is misspelled. Carefully review the spelling of the name and correct it.
 - The secret was deleted accidentally.

- **Cannot find volume.** Output similar to this phrase can indicate the following issues and remedies:
 - The volume information is incorrect. Carefully review the information and correct it.
 - The volume information refers to something that does not exist. For example, the NFS server that is specified in an NFS volume does not exist. Review the availability of the items specified in the volume information and make any applicable corrections.
 - The volume refers to a PVC that does not exist or is unbound. Verify that the PVC was created correctly.

Diagnose the KILLED Job Status

The KILLED job status indicates that the Compute service terminated the pod creation because it took too long to start. No error output is available for the SAS Workload Orchestrator job. The only available information is Kubernetes events:

```
kubectl -n namespace get events
```

where *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).

When you review the Kubernetes events, look for the following issues:

- The container took too long to be pulled to the node (that is, the pod stays in the PodInitializing state for a long time).

Use `sas-prepull` to pull the container to the node ahead of time. For more information, see [“Configure SAS Image Staging” in SAS Viya Platform: Deployment Guide](#).

- There are not enough resources for the SAS Workload Orchestrator job request. Therefore, the job remained in the Pending state too long.

Allocate more resources or change the policy to limit the resources for other jobs. See [“Concepts” in SAS Viya Platform: Workload Management](#).

Check the Status of the Init Containers

Whether you can view applicable Kubernetes events depends on these limitations:

- The status of the init containers
- The persistence of the Kubernetes events

Note: By default, Kubernetes retains these events for the last hour only. However, the retention value is a configurable parameter. See `--event-ttl` at [Kubernetes: kube-apiserver](#)

Use the applicable command for your situation:

The Compute server pod still exists.

- **Compute server pod is stuck in an init container:** Use this command:

```
kubectl -n namespace describe pod pod_name
```

Where:

- *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).
- *pod_name* is the Compute server pod that you want the command to affect.

By using this command, you can check the following information:

- the events associated with the pod
- the status of each container within the pod by looking at the **State**, **Ready**, and **Restart Count** fields.
- **The init container state is not Ready:** View the log for that specific init container to determine why a failure occurred. You can get the init container logs if the pod has not been killed. Use this command:

```
kubectl -n namespace logs pod_name -c init_container_name
```

where *init_container_name* is the init container that you want the command to affect.

The Compute server pod is gone.

Use the `kubectl get events` command or OpenSearch Dashboards to view log messages. See [“Filter SAS Compute Server Log Messages” in SAS Viya Platform: Using SAS Viya Monitoring for Kubernetes](#).

Verify the UID and GID Values

Compute servers run as a specified user. That user's UID and GID values are used in the pod to access mounted file systems. When initializing the pod object information, the Launcher service completes these tasks:

- Gets the user's UID and GID values from the Identities service.
- Validate the values are set up correctly.

Therefore, you should verify that the user's UID and GID values are set correctly.

- If the user's UID and GID values are not set, a Compute server does not start.
- If the user's UID and GID values are set incorrectly, that user's ability to see the files and directories on the file system is impacted.

See [“How to Manage UIDs and GIDs with the Identities Service” in SAS Viya Platform: Identity Management](#).

Investigate a Failure Related to SAS Launcher Service

SAS Launcher Service and Job Failure

When troubleshooting SAS Launcher service, be aware of its role in deleting jobs. Launcher service creates and deletes jobs in these situations:

SAS Compute server creation.

When SAS Compute server is created by SAS Launcher service, the SAS Launcher service retrieves the relevant PodTemplate (by default, `sas-compute-job-config`) and generates a Kubernetes job manifest.

- **SAS Workload Orchestrator is not in use.** The Kubernetes job manifest is passed to the Kubernetes API. The API creates a Kubernetes job object.
- **SAS Workload Orchestrator is in use.** The Kubernetes job manifest is passed to SAS Workload Orchestrator that creates a pod.

SAS Compute server deletion.

- **SAS Workload Orchestrator is not in use.** When the Compute server pod fails, Launcher service deletes the Kubernetes job object. However, that deletion might fail to happen and the Kubernetes job object persists.
- **SAS Workload Orchestrator is in use.** When the Compute server pod fails, Launcher service marks the SAS Workload Orchestrator job as completed or killed, depending on the situation. However, there can be a failure to mark the job, but the Launcher service process record is still removed. The SAS Workload Orchestrator job persists.

A persistent job (either Kubernetes or SAS Workload Orchestrator) can cause these problems:

- **Namespace Pollution.** When job deletions fail, jobs accumulate and clutter the namespace. The clutter makes it difficult to manage and troubleshoot other resources.
- **Increased Storage Usage.** Persistent jobs consume storage resources. It can add up and affect cluster performance.

Determine If SAS Launcher Service Received a Request to Run a Compute Server

To determine whether Launcher service received any job submissions to start Compute server, view the log messages for all Launcher service pods. Use your preferred log-messages software or this command:

```
kubectl -n namespace logs -l app=sas-launcher
```

where *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).

TIP This command shows only the last 10 lines of output from the SAS Launcher logs. To show more lines, add the `--tail` option to the command. To show 50 lines, specify `--tail=50`. To show all lines, specify `--tail=-1`.

In the output, look for a message similar to this example:

```
Process ID "ab63d606-6438-4c3b-b613-8551ca1d19ba" was created by the user: "sasadm"
```

Determine whether the Launcher process ID in the message matches the Launcher process ID that is part of the Compute server pod name. See [“Decipher the Compute Server Pod Name” on page 22](#).

If Launcher service received a process request that matches Compute server, go to [“View the SAS Workload Orchestrator Job Status and Error Output” on page 28](#).

If Launcher service did not receive a process with the same name as the Compute server, then the launch request to SAS Launcher failed. Review the SAS Launcher service logs for relevant errors or warnings. If none are found, see [“Examine the Compute Service Logs” on page 24](#).

Diagnose a Compute Server Time Out

Typically, when a Compute Server times out, it is because the Kubernetes pod failed to start within the expected range of time. Review the Kubernetes events to help determine whether the Kubernetes pod is the root cause. Use this command:

```
kubectl -n namespace get events
```

where *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).

When you review the events, look for output that is similar to the following phrases.

TIP In this section, the problem might be due to an accidental deletion. You can remedy accidental deletions by redeploying the assets associated with the deleted item. Use the original deployment method (such as kubectl commands).

- **Container does not exist.** Output similar to this phrase can indicate the following issues and remedies:
 - The name of the container image is spelled incorrectly. Carefully review the spelling of the name and correct it.
 - The container image is no longer in the repository. Typically, this is due to a newer version replacing the container image. Identify and use the name of the newer container image.
- **Cannot pull container due to permissions problem.** Output similar to this phrase can indicate the following issues and remedies:
 - The image-pull secret specified in the pod template does not exist.
 - During an update to a newer release of SAS Viya platform, the secret was updated and the hash in its name changed, but the pod template was not updated to use the new name. Update the pod template to use the new name of the secret.
 - The secret name is misspelled. Carefully review the spelling of the name and correct it.
 - The secret was deleted accidentally.
 - The image-pull secret exists, but does not allow authentication.
Make sure that the image pull secret has correct information in it.
- **Cannot find configmap.** Output similar to this phrase can indicate the following issues and remedies:
 - During an update to a newer release of SAS Viya platform, the config map was updated and the hash in the name changed, but the pod template was not updated to use the new name. Update the pod template to use the new name of the config map.
 - The config map name is misspelled. Carefully review the spelling of the name and correct it.

- ❑ The config map was deleted accidentally. The kubectl command might not have been applied completely (for example, it was applied to the namespace only). Rerun the commands to restore a consistent state.
- **Cannot find secret.** Output similar to this phrase can indicate the following issues and remedies:
 - ❑ During an update to a newer release of SAS Viya platform, the secret was updated and the hash in the name changed, but the pod template was not updated to use the new name. Update the pod template to use the new name of the secret.
 - ❑ The secret name is misspelled. Carefully review the spelling of the name and correct it.
 - ❑ The secret was deleted accidentally.
- **Cannot find volume.** Output similar to this phrase can indicate the following issues and remedies:
 - ❑ The volume information is incorrect. Carefully review the information and correct it.
 - ❑ The volume information refers to something that does not exist. For example, the NFS server that is specified in an NFS volume does not exist. Review the availability of the items specified in the volume information and make any applicable corrections.
 - ❑ The volume refers to a PVC that does not exist or is unbound. Verify that the PVC was created correctly.

Also, look for the following issues:

- The container took too long to be pulled to the node (that is, the pod stays in the PodInitializing state for a long time).
Use `sas-prepull` to pull the container to the node ahead of time. For more information, see [“Configure SAS Image Staging” in SAS Viya Platform: Deployment Guide](#).
- There are not enough resources for the SAS Workload Orchestrator job request. Therefore, the job remained in the Pending state too long.
Allocate more resources or change the policy to limit the resources for other jobs. See [“Concepts” in SAS Viya Platform: Workload Management](#).

Increase Launch Time-Out Period

If the Launcher service process does not achieve the Running state within the configured `serverLaunchTimeout` period (which defaults to 60 seconds), Compute service sends a request to the Launcher service to kill the process. No information is logged about the cause of the failure to reach the Running state. If this situation occurs, review the Kubernetes events associated with the Kubernetes resource name.

You can modify the server launch time-out period in the following ways:

- Through a compute context. A compute context contains specifications on how the compute starts up. Use the `serverLaunchTimeout` attribute to set the time-out period to something higher than the default value of 60 seconds. See [“Create a Compute Context” in SAS Environment Manager: User’s Guide](#).

Table 1 *serverLaunchTimeout Compute Attribute*

Attribute	Value	Details
serverLaunchTimeout	Default value is 60. (Unit is always seconds.)	Specifies how long the service waits for the new server to start up and reach a running state. The value of the attribute must be greater than zero seconds. If specified in the compute context request, the value of the attribute is used by all the session requests for that context. Specifying a different value in the session request should not override the context's value. If specified in the session request, the value of the attribute is used for only that single session request. If the attribute is also specified in the compute context request, then the value in the context has the precedence over the value that is specified in the session.

To view all available compute attributes, see [“Compute Attributes” in SAS Viya Platform: Programming Run-Time Servers](#).

- You can increase the launcher time-out for all compute contexts by using a command similar to the following:

```
kubectl exec -it sas-rabbitmq-server-0 -- /opt/sas/viya/home/bin/sas-bootstrap-config
kv write --force -- config/compute/sas.compute.server.LaunchTimeout
launch_timeout_in_seconds
```

Run the following command to verify the change:

```
kubectl exec -it sas-rabbitmq-server-0 -- /opt/sas/viya/home/bin/sas-bootstrap-config
kv read --recurse config/compute
```

Monitoring SAS CAS Server

The following information can help you monitor and troubleshoot the CAS server. If you have the SAS Viya Monitoring for Kubernetes project deployed, use the SAS CAS Overview Dashboard to monitor the CAS server. See [“Monitoring SAS CAS Server” in SAS Viya Platform: Using SAS Viya Monitoring for Kubernetes](#).

About the CAS Server

SAS Cloud Analytics Services (CAS) is the primary analytic engine in the SAS Viya platform. The CAS server provides a run-time environment for data management and analytics. Sessions are used to enable clients to communicate with the server in order to request actions. Here are some examples of clients:

- SAS Studio, which provides a programming environment for developing and submitting SAS programs to the CAS server
- SAS Visual Analytics, which lets you create and view interactive reports, explore data, and discover hidden insights with advanced analytics
- open-source software such as Python, Lua, and R to run data analysis on the CAS server

Because CAS is engineered to use all available resources to complete each request, it can drive the available CPU core to 100% utilization. It also consumes memory to process large volumes of data as quickly as possible, as CAS is an in-memory server.

See Also

- [“SAS Cloud Analytic Services: Concepts” in SAS Viya Platform: SAS Cloud Analytic Services](#)
- [Welcome to SAS Studio](#)
- [Welcome to SAS Visual Analytics](#)
- [“Memory” in SAS Cloud Analytic Services: Fundamentals](#)
- [Learning the SAS Viya Platform](#)

SAS CAS Server Architecture

Before you begin to monitor your CAS server, be aware of your CAS server topology, the number of users who interact with CAS, and the volume size of user data. Use the following checklist as a guide:

- ☐ **Is your CAS server SMP or MPP?** The CAS server can run on a single node or as a distributed server on multiple nodes. These two types of architecture have the following distinctions:
 - **Massively Parallel Processing (MPP) multi-nodes** where three roles are assigned to individual pods for CAS:
 - One controller: When the server starts, the controller process is started. This process is sometimes referred to as the server controller. The controller accepts connections from clients.
 - One or more workers: The CAS server has multiple pods that are assigned to the worker role. The controller parses out work to each worker pod. Each worker pod sends the results of its computations back to the controller.
 - One backup controller (optional): This provides fault tolerance for the CAS controller.
 - **Symmetric Multi-processing (SMP) single node** where there is only one pod for CAS. The server acts as the controller, and because there are no worker nodes, the controller pod performs the data analysis on the rows of data that are in memory.
- ☐ **Do you have multiple CAS servers?**

TIP To find your current CAS server topology, run the following `kubectl` command:

```
export viya=viya_namespace
kubectl -n $viya get casdeployments
```

For similar commands, see [“SAS Viya Pods: Cloud Analytic Server \(CAS\)” on page 51](#).

- ☐ **How many users are interacting with the CAS server throughout the day? (How many concurrent sessions during a specified time period?)** When resources are provisioned correctly, there can be many concurrent CAS sessions and your CAS server runs without interruption.
- ☐ **What are the average and maximum sizes of data sets that users load and manipulate?** These sizes might affect the amount of disk space that is required for SAS temporary data or the CAS disk cache.
- ☐ **Which method is used for CAS workload placement on nodes in your cluster?** See [“Workload Placement Strategy” on page 38](#).

See Also

- [“Architecture” in SAS Cloud Analytic Services: Fundamentals](#)
- [“CAS Server Resources” in System Requirements for the SAS Viya Platform](#)
- [“Multiple CAS Servers” in SAS Viya Platform: SAS Cloud Analytic Services](#)

Workload Placement Strategy

CAS server pods have CPU and memory **requests** in their container’s templates. The kube-scheduler uses this information to decide on which node to place the pod. The resource **limits** are also

specified for a container. The kubelet, a Kubernetes agent that is running on each node, enforces those limits so that the running container is not allowed to use more of that resource than the limit that is set.

By default, CAS operates with auto-resourcing enabled, which allows the server to dynamically adjust resource usage. The CAS Operator scans node specifications and allocates 95% of the available CPU and memory to CAS worker pods. When you use auto-resourcing, CAS must be hosted on node pools that are fully dedicated to CAS. Furthermore, the dedicated node pools should be equipped with high-performance hardware. This method is recommended for Cloud-based deployments and virtualized environments, where CAS must maximize its resource utilization without manual intervention.

TIP Have you placed your CAS server on node pools that are fully dedicated to CAS? Use the following command to verify labels and taints on your nodes:

```
kubect1 get nodes -o=custom-  
columns=NAME:.metadata.name,LABELS:.metadata.labels,TAINTS:.spec.taints
```

The alternative is to manually set resource requests and limits, enabling the Kubernetes scheduler to assign CPU and memory limits to CAS pods. CAS pods can share the node pool with other SAS Viya platform components, such as Compute Server pods. This method allows CAS to operate with guaranteed [quality of service \(QoS\)](#) to prevent pods from being evicted and to maintain performance consistency. This method is suitable for bare-metal environments where resource flexibility is required, or where infrastructure costs need to be optimized.

TIP Have you manually set resource requests and limits, enabling the Kubernetes scheduler to assign CPU and memory limits to CAS pods? Run the following command to display the resource definitions for all the containers of a given pod:

```
export viya=viya_namespace  
  
kubect1 -n $viya get pod sas-cas-server-default-controller -o json | jq  
".spec.containers[].name, .spec.containers[].resources"
```

See Also

- [“Tuning Suggestions for Physical Machines” in SAS Viya Platform Administration: Tuning](#)
- [“Plan the Workload Placement” in SAS Viya Platform: Deployment Guide](#)
- [“Adjust RAM and CPU Resources for CAS Servers” in SAS Viya Platform: Deployment Guide](#)
- [“Sizing CAS Server” in SAS Viya Platform: SAS Cloud Analytic Services](#)
- [“Tuning the CAS Server” in SAS Viya Platform Administration: Tuning](#)

Describe Your SAS Viya Platform Deployment

Introduction

Use the following document to describe your SAS Viya platform deployment through the `kubectl` Kubernetes command-line interface. For the `kubectl` version requirement, see [“Kubernetes Client Machine Requirements” in *System Requirements for the SAS Viya Platform*](#).

With this knowledge, you can monitor, troubleshoot, and modify your deployment in order to maintain a fully functioning and highly performant environment.

About Elevated Kubernetes Permissions

Because the SAS Viya platform is deployed to a Kubernetes cluster, any `kubectl` commands that you run in the cluster require that you have cluster-wide or namespace access. Kubernetes uses the role-based access control (RBAC) method. For more information about RBAC authorization, see [Kubernetes documentation](#).

For an overview of cluster-wide resources that are used by the SAS Viya platform and custom SAS Viya API groups, see [“Cluster Resources and Roles That Require Elevated Permissions” in *System Requirements for the SAS Viya Platform*](#).

TIP Run `kubectl api-resources` to generate a full list of Kubernetes resources, and if the resource is namespaced.

Set the \$viya Environment Variable

The `$viya` environment variable identifies the SAS Viya namespace and it is used on `kubectl` commands that require the `--namespace viya_namespace` flag. Make sure that the environment variable is created before you use the reference in your `kubectl` commands. Here is an example:

```
export viya=viya_namespace
echo "The SAS Viya namespace is $viya"
kubectl get pods -n $viya
```


As an alternative, you can set the SAS Viya namespace to act as the current namespace and eliminate the `--namespace` or `-n` flag when you use `kubectl` commands to get information about your SAS Viya platform deployment. For the command to use, see [“About the Current Namespace” on page 60](#).

SAS Viya Deployment Release

- ☐ **Print the release cadence of your deployed SAS software.** SAS offers two release cadences so that the latest features and fixes can be applied at different intervals depending on business needs. Stable releases are available monthly and Long-Term Support releases are available once every six months. Also, selected critical bug fixes and security vulnerability fixes are delivered as patch updates as soon as they are ready. See [“Release Schedule and Versions” in *Getting Started with SAS Viya Platform Operations*](#).

To determine the cadence, release, and version of the software that is currently deployed, use one of the following options:

- From the `checksums.txt` file that is located in your deployment assets directory, run the following command:

```
export DEPLOY=directory_of_Viya_deployment_assets
cat $DEPLOY/sas-bases/checksums.txt | grep Cadence
```

- In a SAS web application in which you are signed in, view the **About** box. It contains the software version and release.
- From within your Kubernetes cluster, run the following command:

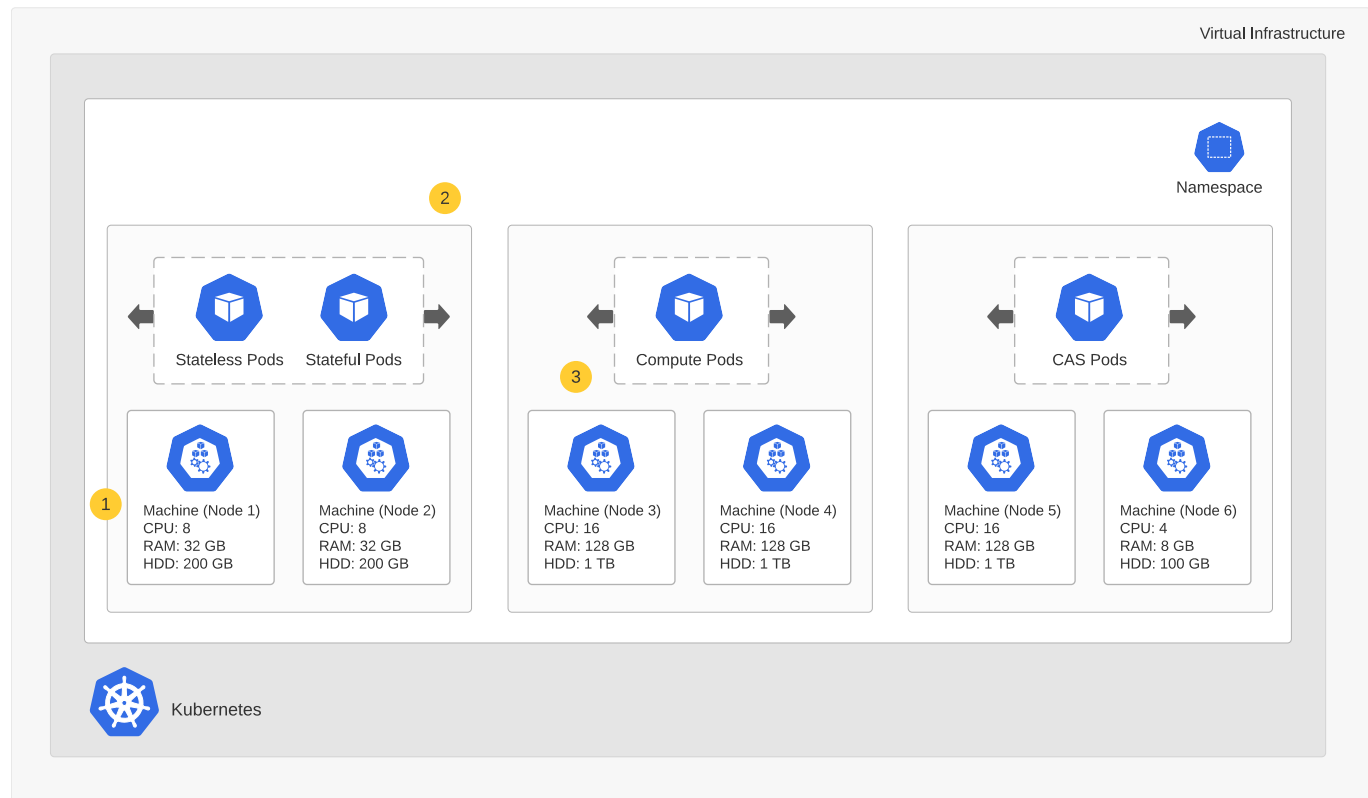
```
export VIYA=Viya_Namespace
kubectl -n $VIYA get cm -o yaml | grep ' SAS_CADENCE'
```

- ☐ **Use the Update Checker Report.** Using the Update Checker is a best practice. It provides a report that enables you to track and deploy software updates. See [“Update Checker Report” in *SAS Viya Platform Operations: Updating Software Tasks*](#) for more information. For the commands to use, see [“View the Latest Report” in *SAS Viya Platform Operations: Updating Software Tasks*](#).
- ☐ **Plan your SAS Viya platform software updates.** See [SAS Viya Platform Operations: Updating Software Tasks](#).

There are many considerations when updating your software. Be sure to review [“Related Topics for Updating Software” in *SAS Viya Platform Operations: Updating Software Tasks*](#) and [“Best Practices for Updating Software” in *SAS Viya Platform Operations: Updating Software Tasks*](#).

Nodes in Your Kubernetes Cluster

The following diagram illustrates how pods can land on nodes based on workload management. Node pools are dedicated to a workload class, which is identified by the work that is performed by the associated pods. Note that not all components and options are depicted.



- 1 A Kubernetes cluster consists of two types of nodes: one or more control plane nodes and a set of worker nodes that run containerized applications. Nodes can be a physical or virtual instance of a server and can vary in size. The *kube-scheduler* from the control plane selects the optimal node to run pods.

For information about the Kubernetes infrastructure requirements, including node requirements, see [“Virtual Infrastructure Requirements” in System Requirements for the SAS Viya Platform](#).

Note: If your SAS Viya Platform is deployed in a cloud environment, which includes supported versions of Microsoft Azure, Amazon Web Services, and Google Cloud, the control plane components are not visible to administrators.

Note: Some cloud providers use *node pools* or *node groups* to refer to a collection of *dynamic* nodes.

- 2 A pod represents a unit of deployment and encapsulates an application's container and related storage resources. (A pod can hold multiple containers to provide a set of functions, and a pod might also have one or more init containers, which run before the app containers are started.) Kubernetes schedules pods to run on nodes based on scheduling principles. The scheduler filters existing nodes according to the specific container or pod requirements. Factors for scheduling decisions are resource requirements, hardware, software, or policy constraints, affinity and anti-affinity specifications, data locality, inter-workload interference, and so on.
- 3 SAS Viya pods have resource specifications that are based on the type of work that they perform. Labels are used to identify five groups of SAS Viya pods:
 - **stateless:** web applications and microservices.

- **stateful:** open-source components, such as Postgres, Consul, RabbitMQ, Redis.
- **cas:** SAS Cloud Analytic Services (CAS), the primary compute engine.
- **compute:** compute processes, such as SAS Studio sessions or batch jobs.
- **connect:** SAS/CONNECT provides the ability for clients to launch SAS compute processes within a local or remote SAS environment.

Use the following Kubernetes commands as examples for observing or validating your environment throughout its life cycle.

☐ **Collect information about the nodes in your cluster with kubectl commands. Here are some examples:**

```
kubectl get nodes -o wide
```

Where `-o wide` option returns more information about the node, such as OS image, internal IP address of the node, kernel version and container runtime.

```
kubectl describe node name_of_node
```

The following information is returned for the node:

- Capacity CPU and memory
- Allocatable CPU and memory
- Number of pods scheduled on node
- A list of each pod that includes the namespace to which it belongs and the pod's CPU requests and limits and memory requests and limits
- Allocated resources

TIP Use the [SAS Viya Administration Resource Kit](#) to produce the SAS Viya Platform Deployment Report. The report contains a list of SAS deployed components with application resources grouped together that are deployed in your Kubernetes cluster.

TIP The SAS Pre-Install Check Utility is also available in the SAS Viya 4 ARK GitHub project. A pre-deployment assessment report is generated to verify that your cluster meets the documented requirements to deploy SAS Viya. See [System Requirements](#).

☐ **Verify labels and taints on your nodes.**

```
kubectl get nodes -o=custom-
```

```
columns=NAME:.metadata.name,LABELS:.metadata.labels,TAINTS:.spec.taints
```

Labels and taints are required if you are implementing a workload placement strategy of your SAS Viya pods. For information see [“Plan the Workload Placement” in SAS Viya Platform: Deployment Guide](#).

SAS Viya Namespaces

Each SAS Viya deployment requires a dedicated namespace.

☐ **List the namespaces in your Kubernetes cluster.**

```
kubectl get namespaces
```

☐ **Print the configMap that contains all metadata about the SAS Viya version that is deployed.**

```
kubectl describe configmap -l "orchestration.sas.com/lifecycle==metadata"
```

☐ **Run the following command to list the SAS Viya cadence name, version and release of your deployment.**

```
kubectl get configmap -l "orchestration.sas.com/lifecycle==metadata" --output
jsonpath="{.items[*].data}" | jq | grep "SAS_CADENCE"
```

TIP For information about jq, see [“About Kubectl Command Output” on page 60](#).

To return information from a `kubectl` command about SAS Viya objects in a namespace, you can perform one of the following options:

- Include the `--namespace` or `-n` flag on your `kubectl` command.
- Set the current namespace to your SAS Viya namespace. Then you do not need the `--namespace` or `-n` flag on your command. See [“About the Current Namespace” on page 60](#).
- Use the `$viya` environment variable on your `--namespace` or `-n` flag. See [“Set the \\$viya Environment Variable” on page 40](#).

Note: This document uses the `$viya` environment variable that contains your SAS Viya namespace on the `--namespace` or `-n` flag. Make sure that the environment variable is created before you use `$viya` in your `kubectl` commands. Here is an example:

```
export viya=viya_namespace
echo "The SAS Viya namespace is $viya"
```

SAS Viya Pods Overview

Pods are the environment for running the SAS Viya containers. Kubernetes schedules pods to run on nodes based on scheduling principles. The scheduler filters existing nodes according to the specific requirements for the container, pod, or both. Scheduling decisions can be affected by the following factors:

- resource requirements
- hardware, software, and policy constraints

- affinity and anti-affinity specifications
- data locality
- inter-workload interference
- and more

After a pod is scheduled to run on a node, the pod remains on that node until one of the following situations occur:

- The pod finishes execution.
- The pod object is deleted.
- The pod is evicted for lack of resources.
- The node fails.

Note: The [\\$viya environment variable on page 40](#) is used on `kubectl` commands that require the `--namespace` flag. Make sure that the environment variable is created before you use these `kubectl` commands.

☐ Print a list of SAS Viya pods.

```
kubectl -n $viya get pods -o wide
```

The `-o wide` flag returns the node name on which the pod is running and the pod IP. The `READY` column contains the pod container status, where the first number is the current number of ready containers and the second number is the expected number of containers in the pod.

Pods should have a status of **Running** or **Completed**. The output also shows how the SAS Viya pods are distributed across the nodes in your cluster.

☐ Print a count of how many pods have the `sas.com/deployment=sas-viya` label and suppress the column headings .

```
kubectl get pods --selector 'sas.com/deployment=sas-viya' --no-headers | wc -l
```

Because there are so many pods in your SAS Viya namespace, use labels to identify the SAS Viya pod or key groups of SAS Viya pods. Each pod has a unique label that can be used to reference a pod. Use the `-l` or `--selector=` flag to filter pod instances. Here is an example:

```
kubectl describe pod -l app=sas-identities
```

Note: You can run the following command to view the labels of all of your SAS Viya pods. However, the output is long:

```
kubectl -n $viya get pods --show-labels
```

☐ Print details for a single pod.

```
kubectl describe pod pod_name -n $viya
```

or:

```
kubectl describe pod -l app=pod_label -n $viya
```

☐ Print resource usage of all SAS Viya pods.

```
kubectl top pod --containers-n $viya
```

By default, the `kubectl top pod` command returns resource usage for all pods. If you pass a specific pod instance name to the command, you can obtain the information for the specified

pod instance only. Use the `-l` or `--selector=` flag to filter pod instances. Here is an example that specifies the internal crunchy Postgres:

```
kubectl top pod -l "postgres-operator.crunchydata.com/data==postgres" --containers -n $viya
```

Use labels to identify key groups of SAS Viya pods. They are grouped based on shared resource specifications that depend on the type of work that they perform. The following list shows the five groups of SAS Viya pods:

- *stateless*: web applications and microservices
- *stateful*: open-source components, such as Postgres, Consul, RabbitMQ, Redis
- *cas*: SAS Cloud Analytic Services (CAS), the primary compute engine
- *compute*: compute processes, such as SAS Studio sessions or batch jobs
- *connect*: SAS/CONNECT provides the ability for clients to launch SAS compute processes within a local or remote SAS environment

See “Plan the Workload Placement” in *SAS Viya Platform: Deployment Guide* for details.

To view a subset of SAS Viya pods, use the following options:

- label selector flag: `--selector` or `-l`
- **grep** command: pipe (`|`) the output from the `kubectl` command to a `grep` command in order to search and manipulate text patterns in the output. Piping is a form of redirection. The `STDOUT` stream (*output*) from one command becomes the `STDIN` stream (*input*) to the command that follows the pipe (`|`).

TIP Use Linux commands to manipulate the output of your `kubectl` command. Here is an example of returning the status of critical pods on which other pods rely:

```
kubectl -n $viya get pods | egrep -i 'crunch|cas-|rabbit|consul|auth|compute|logon|cache|sas-data-server-operator|redis|sas-pyconfig|workload'
```

TIP Redirect the output from one or more commands to a file by using the redirection syntax: `>`.

SAS Viya Pods: Stateful Services

Stateful applications are managed by the Kubernetes StatefulSet workload API object. Pods of the `statefulSet` type retain their state, role, and a persistent identifier across rescheduling. These pods are labeled with the workload class of `stateful` and they are deployed in a high availability (HA) configuration by default. (The label assignment and the number of replicas are specified in the container images of the SAS software.)

Note: The [\\$viya environment variable on page 40](#) is used on `kubectl` commands that require the `--namespace` flag. Make sure that the environment variable is created before you use these `kubectl` commands.

☐ **Print a list of pods with the workload class label of 'stateful'.**

```
kubectl get pods -n $viya -o wide -l workload.sas.com/class=stateful
```

As an alternative, you can use the following commands:

```
kubectl get statefulsets -n $viya
```

```
kubectl get pods -n $viya | grep -E "consul|rabbit|redis|postgres|crunch"
```

The output from the above commands includes the following SAS infrastructure servers:

Pod Name	Description
sas-consul-server- [0-2] where 0-2 represents 3 replica pods (HA).	SAS Configuration Server acts as a central repository for configuration data that is used by SAS Viya platform microservices at start-up to load default values and to discover any service dependencies. For information about scaling SAS Configuration Server up or down, see “SAS Configuration Server” in SAS Viya Platform: Infrastructure Servers .
sas-crunchy-platform-postgres-00-hash-0 (3 replica pods) sas-crunchy-platform-postgres-backup-hash sas-crunchy-platform-postgres-repo-host-0	The SAS Infrastructure Data Server stores SAS Viya platform user content, such as reports, authorization rules, selected source definitions, attachments, and user preferences. For more information, see “SAS Infrastructure Data Server (Platform PostgreSQL)” in SAS Viya Platform: Infrastructure Servers . Note: You might have deployed an external PostgreSQL database. See “PostgreSQL Server Requirements” in System Requirements for the SAS Viya Platform . Use the <code>kubectl</code> command at the end of this section to determine if you have an internal or external Postgres.
sas-rabbitmq-server- [0-2] where 0-2 represents 3 replica pods (HA).	SAS Message Broker is part of the event-driven architecture in which SAS Viya platform services participate. It is configured with 3 replicas by default (HA). For information, see “SAS Message Broker” in SAS Viya Platform: Infrastructure Servers .
sas-redis-server- [0-1]	Redis provides a distributed cache technology to microservices in SAS Viya platform. Redis is started in cluster mode and one single Statefulset with two pods, one controller and one replica node. (Starting with 2024.12, the SAS Redis Operator is deprecated. The only function of the SAS Redis Operator is to remove resources that were previously created by the operator. It will be removed in a future release.) For information, see “SAS Redis Server” in SAS Viya Platform: Infrastructure Servers .

The output from the above commands includes other SAS Viya servers that have the label of `stateful` workload class. This depends on your software order, your deployed cadence version, and your decision to enable the resource. Here are examples:

Pod Name	Description
<code>sas-crunchy-cds-postgres-00-hash-0</code> (5 containers in each pod), a completed back-up pod, and a repo host pod.	The CDS PostgreSQL Server is required by some SAS Viya products.
<code>sas-airflow-redis-0</code> <code>sas-airflow-worker-0</code> (with 2 containers)	SAS Risk Solutions use an Apache Airflow database. This is new starting at LTS 2024.03 (May 2024): “New SAS Risk Solutions Available” in What’s New in SAS Viya Platform Operations . For more information, see “Requirements for SAS Risk Solutions” in System Requirements for the SAS Viya Platform .
<code>sas-data-agent-server-colocated-0</code>	Cloud Data Exchange for the SAS Viya Platform includes the co-located SAS Data Agent and the remote SAS Data Agent. For more information, see “Overview” in Cloud Data Exchange for the SAS Viya Platform: Administrator’s Guide .
<code>sas-opendistro-default-0</code>	An instance of OpenSearch is required for a deployment of the SAS Viya platform. If you decide to deploy an internal instance of OpenSearch, SAS provides the instance and configures the deployment for you. For more information, see “OpenSearch Requirements” in System Requirements for the SAS Viya Platform .
<code>sas-workload-orchestrator-[0-1]</code>	By default, there are two SAS Workload Orchestrator manager pods. They are a <code>statefulSet</code> Kubernetes resource that have the <code>workload.sas.com/class=stateful</code> label.

SAS Viya Pods: Stateless Services

The SAS Viya platform contains many services often referred to as microservices. A *microservice* is a discrete service that runs in its own process and that communicates using HTTP. Configuration data for these microservices is stored in SAS Configuration Server, such as JVM options, logging levels, and much more. See [“Service Configuration Properties: Reference” in SAS Viya Platform: Configuration Properties](#). (Stateless services are managed by the Kubernetes Deployment workload API object.)

Note: The [\\$viya environment variable on page 40](#) is used on `kubectl` commands that require the `--namespace` flag. Make sure that the environment variable is created before you use these `kubectl` commands.

These pods are labeled with the workload class of `stateless`. Most of these stateless services are not configured for high availability (HA) by default. The label assignment and the number of replicas are specified in the container images of the SAS software.

☐ **Print a list of pods with the workload class label of stateless.**

```
kubectl get pods -n $viya -o wide -l workload.sas.com/class=stateless
```

☐ **Print the number of stateless service pods.**

```
kubectl get pods -n $viya -o wide -l workload.sas.com/class=stateless | wc -l
```

☐ **Print stateless pods that are in high availability (HA).**

```
kubectl -n $viya get hpa
```

By default, almost all stateless services are deployed with only one replica. You might see an exception, or services might have been configured for HA at post-deployment. For information, see [“Configure High Availability” in SAS Viya Platform: Deployment Guide](#). Be aware that configuring HA for all of your microservices doubles the number of pods that are deployed. Verify that the cluster has enough resources.

☐ **List services that are written in Java.**

```
kubectl -n $viya get deployments --selector='sas.com/deployment-base=spring'
```

For services written in Java, changes trigger a refresh and no restart of the service is required. Verify that the configuration changes were applied. Restart the service if the change was not applied.

Java services might require adjustments to JVM options. For example, Java heap size should be 75% of pod memory limits. To view jvm settings for a service, you can use SAS Environment Manager. Select **Configuration page** ⇒ **Definitions** ⇒ **jvm**. Select a service from the right pane.

☐ **List services that are written in Go.**

```
kubectl -n $viya get deployments --selector='sas.com/deployment-base=golang'
```

A restart is required if changes are made to a configuration instance of a Go service. Typically, the restart occurs automatically.

☐ **Run the following command to restart a service.**

```
# Find name of service.
kubectl get pods -n $viya | grep sas-service_name_without_hash
# Copy service into the following command.
kubectl delete pods -n $viya sas-service_name_with_hash

kubectl delete pods -n $viya -l app=sas-service_name_without_hash
```

For more information, see [“Managing a Specific Server or Service” in SAS Viya Platform: General Management of Servers and Services](#).

SAS Web Applications are labeled as stateless services.

☐ **Find SAS web application pods.**

```
kubectl -n $viya get ing | grep -e -app
```

☐ **Print SAS web application URLs. Here is an example that uses SAS Drive:**

```
STUDIO_URL=$(kubectl -n ${viya} get ing sas-studio-app -o custom-
columns='hosts:spec.rules[0].host' --no-headers)/SASStudio/"
echo $STUDIO_URL
```

SAS Viya Pods: Programming Run-time Servers

A programming run-time environment includes several SAS Viya platform servers. For information about these servers, see [“Programming Run-Time Servers: Overview” in SAS Viya Platform: Programming Run-Time Servers](#). For the general process of how the SAS Compute Server works, see [“How Compute Server Works” in SAS Viya Platform: Programming Run-Time Servers](#).

Note: The [\\$viya environment variable on page 40](#) is used on `kubectl` commands that require the `--namespace` flag. Make sure that the environment variable is created before you use these `kubectl` commands.

- ☐ **Find services that are responsible for starting a Compute session or a job that executes SAS code. Note that these services are always running.**

```
kubectl get pods -n $viya -o wide -l 'app=sas-compute'
kubectl get pods -n $viya -o wide -l 'app=sas-batch'
kubectl get pods -n $viya -o wide -l 'app=sas-connect-spawner'
kubectl get pods -n $viya -o wide -l 'app=sas-connect'
kubectl get pods -n $viya -o wide -l 'app=sas-launcher'
```

The following table lists these services:

Pod Name	Service
<code>sas-compute-hash</code>	SAS Compute service
<code>sas-batch-hash</code>	SAS Batch service
<code>sas-connect-spawner-hash</code>	SAS/CONNECT spawner
<code>sas-connect-hash</code>	SAS/CONNECT service
<code>sas-launcher-hash</code>	SAS Launcher service

The following examples are common:

- Clients such as SAS Studio use the Compute service to start a compute session.
 - Interactive tools such as `sas-viya` CLI or REST API use the batch service to start a batch job.
- ☐ **The SAS Launcher service launches the following servers based on the service that made the request (Compute, Batch, or Connect).**

Pod Name	Server
<code>sas-compute-server-hash</code>	SAS Compute server
<code>sas-batch-server-hash</code>	SAS Batch server
<code>sas-connect-server-hash</code>	SAS/CONNECT Server

Note: These servers use the same `sas-programming-environment` container image, but they are configured differently depending on the service and launcher context. The SAS Launcher service constructs a Kubernetes job request with the PodTemplate information to generate the container that is launched as a pod. The container in the pod performs the SAS processing.

☐ **Print a list of pods with the workload class label of 'compute'.**

```
kubectl get pods -n $viya -o wide -l workload.sas.com/class=compute
```

A SAS Workload Orchestrator server pod runs on each node where the `workload.sas.com/class` label is `compute`.

☐ **Find compute server pods.**

```
kubectl -n $viya get pods -o wide -l "launcher.sas.com/job-type in (compute-server)" -L launcher.sas.com/requested-by-client -L launcher.sas.com/username --sort-by=.status.startTime
```

Compute servers are not running in the environment until there is a job request and a server is started by the Launcher service. Options are used to generate more details. In this example, label queries, `(-l)` are used to filter, and label columns `(-L)` are used to return more details than just the user's session. This example shows the `REQUESTED-BY-CLIENT` and the `USERNAME` columns.

☐ **View the log of the pod that was launched by a user.**

```
kubectl logs $(kubectl -n $viya get pods -l launcher.sas.com/username=name_of_user -L status.phase=Running --sort-by=.metadata.creationTimestamp -o NAME)
```

SAS Viya Pods: Cloud Analytic Server (CAS)

For information about on the SAS Cloud Analytic Server, see [SAS Viya Platform: SAS Cloud Analytic Services](#).

Note: The [\\$viya environment variable on page 40](#) is used on `kubectl` commands that require the `--namespace` flag. Make sure that the environment variable is created before you use these `kubectl` commands.

☐ **Run the following command to list all CAS server pods:**

```
kubectl get -n $viya pods --selector app.kubernetes.io/managed-by=sas-cas-operator
```

Here is an alternative command to list CAS pods:

```
kubectl -n $viya get pods -o wide |{ head -1; grep "sas-cas";}
```

Note: The output depends on the following:

- Whether your CAS server is SMP (symmetric multiprocessing) single node or MPP (massively parallel processing) multi-nodes. (In SMP mode, the `sas-cas-server-default-controller` pod performs the role of a worker node.)
- Whether multiple CAS servers are deployed.

Pod Name	Server
<code>sas-cas-control-hash</code>	The pod that manages CAS servers' configurations and communicates with applications, such as SAS Environment Manager and Consul. (It is used by several services.)
<code>sas-cas-operator-hash</code>	The pod that manages all CAS server instances in a SAS Viya namespace, based on the <code>CASDeployments</code> custom resource. An operator is a way of building an application and driving an application within the Kubernetes cluster.
<code>sas-cas-server-default-controller</code>	The pod that manages CAS servers' configurations and communicates with applications (SAS Environment Manager, for example) and Consul. (It is used by several services.) Note: If your environment is SMP single node, the <code>sas-cas-server-default-controller</code> pod performs the role of a worker node.
<code>sas-cas-server-default-backup</code>	This pod provides fault tolerance for the <code>sas-cas-server-default-controller</code> .
<code>sas-cas-server-default-worker-0-n</code>	When a server is running in massively parallel processing (MPP) mode, in addition to a controller, the server also has multiple pods that are assigned the worker role.

- ☐ **Run the following command to list details about the CAS controller:**

```
kubectl -n $viya describe pods sas-cas-server-default-controller \
| grep "casoperator." | awk -F"/" '{print $2}'
```

The output tells you if you have an SMP or MPP CAS server, metadata information about the CAS server, such as the name, and more.

☐ List the containers in your CAS server pod:

```
kubectl -n $viya top pods sas-cas-server-default-controller --containers
```

Note that the `kubectl top pods` command is typically used to get information about the pod resource consumption. If you use the `--containers` parameter, you return a list of the pod's containers.

☐ List data that is mounted into CAS pods.

```
kubectl -n $viya describe pod -l casoperator.sas.com/node-type=controller
```

There is a lot of output returned with the `kubectl describe` command. Pipe the `kubectl` commands to `jq` commands to return specific information only. For information about `jq`, see ["About Kubectl Command Output" on page 60](#).

Here is an example to list current default CAS server volumes, based on volume type:

```
kubectl -n $viya get pods \
    sas-cas-server-default-controller \
    -o=json \
    | jq '[.spec.volumes[] |
    if has("configMap") then "Name: "+.name, "Type: configMap", ""
    elif has("emptyDir") then "Name: "+.name, "Type: emptyDir", ""
    elif has("hostPath") then "Name: "+.name, "Type: hostPath", "Path:
    "+.hostPath.path, ""
    elif has("nfs") then "Name: "+.name, "Type: nfs", "Path: "+.nfs.path,
    "Server: "+.nfs.server, ""
    elif has("persistentVolumeClaim") then "Name: "+.name, "Type:
    persistentVolumeClaim", "Claim Name: "+.persistentVolumeClaim.claimName, ""
    elif has("secret") then "Name: "+.name, "Type: secret", ""
    else empty end]' \
    | tr -d '",'
```

The following different types of volumes are returned:

- Persistent volumes that are used to store data that need to persist when the pods restart. The type of `persistentVolumeClaim` is used to mount PVs into the CAS server pods. Your output varies, depending on the products that you have licensed, and your PVCs that were set up at deployment. The CAS server includes the following PVCs:
- `sas-commonfiles`: stores CAS Server binaries and files that are required for CAS servers to run. This volume is shared by all CAS server pods in a SAS Viya deployment to help reduce the size of the CAS container in each CAS server pod.
- `cas-default-permstore`: persists the metadata for CAS that includes caslib definitions, permissions, and so on.
- `cas-default-data`: stores data that is saved and possibly reloaded into the CAS server.
- `sas-cas-backup-data`: stores CAS server backups.
- `sas-cas-transfer`: when it is enabled, used for the state transfer.
- Ephemeral volumes that are re-created each time the pod restarts. Examples are `configMap`, `emptyDir`, and `secret`.

☐ List CAS server Persistent Volumes (PV). Persistent Volumes are created and managed at the Kubernetes cluster level. They are a Kubernetes cluster resource, not a namespace resource.

When the `persistentVolumes` resource is queried by using the `kubectl` CLI, the `--namespace` is ignored.

```
kubectl get persistentVolumes -o wide | { head -1; grep "cas-"; }
```

Note that all SAS Viya `persistentVolumes` resource names are prefixed with `pvc-` and the `CLAIM` field contains the `viya_namespace/persistentVolumeClaim` name.

- ☐ **List CAS server Persistent Volumes Claims (PVC).** Persistent Volume Claims are created and managed at the namespace level. They are a namespace resource. When the `persistentVolumeClaims` resource is queried by using the `kubectl` CLI, the `--namespace` flag is important.

```
kubectl -n $viya get persistentVolumeClaims -o wide | { head -1; grep "cas-"; }
```

The `persistentVolumeClaims` create a link between a `persistentVolume`, a Kubernetes cluster resource, and a volume that is defined for a pod in a specific namespace.

Persistent Storage for SAS Viya Software

Kubernetes uses a `PersistentVolume (PV)` API resource for long-term storage in your cluster. Persistent volumes exist beyond the life of containers, pods, and nodes. A pod uses a `PersistentVolumeClaim (PVC)` to get Read and Write access to the persistent volume. PVs are provisioned using a `StorageClass` and PVCs in your SAS Viya deployment take advantage of the `StorageClass`. Your cluster's `StorageClass` is specified in the `storageclass.yaml` file and it is referenced in your `kustomization.yaml` file. See [“Specify PersistentVolumeClaims to Use ReadWriteMany StorageClass” in SAS Viya Platform: Deployment Guide](#).

For an explanation of persistent volumes, see [Kubernetes documentation on persistent volumes](#).

Several SAS Viya components require persistent storage. The SAS Viya deployment process sets up PVCs for pods to claim the necessary storage from the storage class. This in turn creates persistent volumes (PVs) that meet the needs of the claim. See [“Storage Requirements” in System Requirements for the SAS Viya Platform](#) for an overview of what the SAS Viya platform requires.

- ☐ **Print the `StorageClass` that has been defined in your Kubernetes cluster?**

```
kubectl get storageclass
```

Use the output from the above command for the following command:

```
kubectl describe sc name of storage class
```

Note:

- Store the names of all storage classes if you later need to modify storage settings.
- Be aware of the `reclaimPolicy` that is set to **delete** by default. This results in loss of data as corresponding persistentVolumes (PVs) are deleted if you delete a namespace. For more information, see [“Persistent Storage Volumes, PersistentVolumeClaims, and Storage Classes” in System Requirements for the SAS Viya Platform](#).

- ☐ **Print the PVCs in your deployment.**

```
kubectl get pvc -n $viya
```

The [“SAS Viya Pods: Stateful Services”](#) on page 46 have the `stateful` class label and they require persistent storage.

The CAS server requires persistent storage for various CAS data. See [“SAS Viya Pods: Cloud Analytic Server \(CAS\)”](#) on page 51 for commands that return the PVCs for the CAS pods.

Avoid Database Issues

The SAS Infrastructure Data Server component is supported by a PostgreSQL database server. SAS Infrastructure Data Server provides a transactional store that is used by the SAS Viya platform. The server is configured automatically during deployment. However, you have multiple options for optimizing its performance, which can have an outsize impact on overall SAS Viya platform performance. See [“Tuning the PostgreSQL Server”](#) in *SAS Viya Platform Administration: Tuning*.

A database with significant growth can cause problems for the SAS Viya platform. Here are some potential problems:

- Performance is negatively impacted.
- Back up and restore can take a long time.
- Searching through job instances, such as on the **Jobs** page in SAS Environment Manager, can take a long time.
- Compute sessions and jobs can take a long time to start.

To avoid potential problems, see the routine maintenance tasks in [“SAS Infrastructure Data Server \(Platform PostgreSQL\)”](#) in *SAS Viya Platform: Infrastructure Servers*.

See Also

- [“PostgreSQL Server Requirements”](#) in *System Requirements for the SAS Viya Platform*
- [“Delete Job Records”](#) in *SAS Viya Platform: Workload Management*
- [“Deleting Job Objects from the SAS Viya Platform Database”](#) in *SAS Viya Platform: Jobs and Flows*
- [“Setting Job Instances to Expire”](#) in *SAS Viya Platform: Jobs and Flows*

Troubleshoot Command-Line Interface Issues

For detailed information, see [SAS Viya Platform: Using the Command-Line Interface](#).

When troubleshooting the sas-viya command-line interface (CLI), remember the following key points:

- ☐ To diagnose problems with any sas-viya CLI command, use the VERBOSE global option to run the sas-viya CLI in debug mode. See [“Global Option: Verbose” in SAS Viya Platform: Using the Command-Line Interface](#).
- ☐ All users can use the sas-viya CLI, even non-administrative users. However, there are situations where running the sas-viya CLI with administrative permissions might give more optimal results. This is due to the permissions that are required by the underlying functional area, rather than the permissions that are required by the sas-viya CLI itself. For example, suppose that a non-administrator cannot use all of the commands in the identities plug-in to the sas-viya CLI. This might be because they do not have the underlying permissions that are required by the identities service to create identity groups or change group memberships.
- ☐ Commands and subcommands are case sensitive.
- ☐ You must precede an option of a command with two hyphens (--) if the option is a word.

Note: If the option is a single letter, you can precede the option with two hyphens (--) or one hyphen (-). For example, you can use --help or -h for the Help global option.

- ☐ If a single parameter contains spaces (such as an object's name), you must enclose it in quotation marks in order to pass it as one item. Depending on what can be parsed by your operating system, other parameter values might need to be enclosed in quotation marks or escaped accordingly.
- ☐ Linux special characters that are specified on the command line must be preceded (or escaped) with a backslash (\) so that they are not interpreted by the Linux shell. Linux special characters that are included in JSON template files do not need to be escaped.
- ☐ Timestamps that are returned from the CLI are in Universal Time Coordinated (UTC) format rather than in local time. By contrast, timestamps returned from SAS Environment Manager are in local time.

For example, in SAS Environment Manager, you might notice that a SAS Visual Analytics report has a modified date of April 26, 2018 09:23:47. This is the local time. However, the reports plug-in shows that the modified date for the same report is 2018-04-26T13:23:47.314Z. This is UTC time.

- ☐ Use the Help from within each plug-in for information about the available commands, subcommands, and options. See [“Integrated Help” in SAS Viya Platform: Using the Command-Line Interface](#).

Note: The integrated Help supersedes this documentation and provides the most current information about any expanded or enhanced functionality.

See Also

- [“User Is Unable to Log In” in SAS Viya Platform: Using the Command-Line Interface](#)
- [“Command with Option or Global Option Fails” in SAS Viya Platform: Using the Command-Line Interface](#)

Correct a Custom Script Failure

Problem:

A custom script that previously worked now fails. Typically, this failure occurs with custom deployment scripts.

Explanation:

Typically, this failure is due to a pod name change. Pod names might change between releases of SAS Viya platform.

Resolution:

Update the name of the affected pod in the custom script. For details, see [“Pod Changes by Release” in SAS Viya Platform Operations: Updating Software Tasks](#).

Rebalance Pods after Node Failure in HA Environment

Problem:

Following a node failure event, pods do not automatically redistribute back to the original failed nodes once the infrastructure is restored. High Availability (HA) is not in effect at that point.

Explanation:

[High Availability](#) is not achieved when pods do not redistribute back to the original failed node or nodes. This results in an uneven workload distribution and the potential underutilization of available resources.

Resolution:

Manual intervention is currently required to rebalance pods in order to ensure full system resilience. This can be achieved through a planned outage where all pods are stopped and

started. For more information, see [“Starting and Stopping a SAS Viya Platform Deployment”](#) in *SAS Viya Platform: General Management of Servers and Services*.

Recover from a Disaster

Disaster recovery in SAS Viya 4 is the process for recovering in the event of a disaster. Recovery in this context refers to regaining access to data, hardware, and software. For more information, see [“Backup and Restore: Recover from a Disaster”](#) in *SAS Viya Platform: Backup and Restore*.

Feature-Specific Troubleshooting Information

Each of the following links opens the troubleshooting topic for a feature in SAS Viya platform:

Servers, Services, Workload Management

[General Servers and Services](#)

[SAS Cloud Analytic Services](#)

[SAS Compute](#)

[SAS Launcher](#)

[SAS/Connect](#)

[Workload Management, SAS Workload Orchestrator, Queues](#)

[SAS Configuration Server](#)

[SAS Infrastructure Data Server](#)

[SAS Redis Server and SAS Redis Operator](#)

Deployment, Migration, Backup and Restore, Updating

[Deployment Guide](#)

[Full-System Migration from SAS Viya 4](#)

[Full-System Migration from SAS Viya 3.X](#)

[Backup and Restore](#)

[Updating Software](#)

Authentication and authorization

[Authentication](#)

[CAS Authorization](#)

[General Authorization](#)

[Jobs and Flows](#)

[Encryption: Data in Motion](#)

[Identity Management](#)

[SingleStore: Administration and Configuration Guide](#)

[Command-Line Interface](#)

[Configuration Properties](#)

[Mobile](#)

[Fonts](#)

[Licensing](#)

Get Support

Generally, technical support is available to all customers who license SAS software. However, customers are encouraged to engage their designated on-site SAS support personnel as the first support contact. If the on-site SAS support personnel cannot resolve the issue, they should contact SAS Technical Support to report the problem.

Before You Contact Technical Support

Explore the [SAS Technical Support](#) site, which offers access to the SAS Knowledge Base, SAS Communities, Technical Support contact options, and other support materials. If you plan to contact SAS Technical Support, here are some tips that can help:

- Gather the Tech Support Site Number, company name, email address, and phone number. The Tech Support Site Number can be found on the **My Orders** page at My SAS (my.sas.com).
- For problems related to a deployment, go to [SAS® Viya® for Kubernetes Diagnostic Documentation \(KB0038941\)](#) to learn about the get-k8s-info script. The get-k8s-info script collects diagnostic information about the deployment.
- For problems in SAS web applications (such as SAS Environment Manager or SAS Studio), go to [Using the web browser developer tools to investigate issues in SAS® Web Applications \(KB0036430\)](#). This article provides instructions for gathering troubleshooting information.

SAS Technical Support Services and Policies

The SAS Viya platform has two different support policies that are based on which software the customer licenses. SAS recommends that customers install the most current release of their licensed

software products. For more information and guidelines, see [SAS Technical Support Services and Policies for the SAS Viya Platform](#).

Reference

About the Current Namespace

When you issue `kubectl` commands without using the `-n` or `--namespace` flag, then the currently set namespace is used. If no namespace is set, the *default* namespace is used. To identify the currently set namespace, use this command:

```
kubectl config view --minify --output 'jsonpath={..namespace}'
```

To set the current namespace, use this command:

```
kubectl config set-context --current --namespace=namespace
```

where *namespace* is the namespace that you want to be current.

About Kubectl Command Output

Logs are generated in JSON format. The output can be difficult to read. Consider using a tool (such as `jq`) to make the output easily readable. For example, if you use the `jq` command, you can pipe the `kubectl` output to `jq` to parse the result. The following example parses the JSON and also removes non-JSON-formatted lines:

```
kubectl -n namespace logs pod_name | jq -Rr 'fromjson? | [.timestamp,"[",.level,"]",.message] | join(" ")'
```

Where:

- *namespace* is the namespace in which you want to apply the command. If you do not specify this option, the current namespace is used. For details, see [“About the Current Namespace” on page 60](#).
- *pod_name* is the pod that you want the command to affect.

About Compute Server and Job Execution Service Job Names

Typically, when Job Execution Service (JES) submits a job to the Compute provider, a new Compute server is started to run the job. In that case, the job name is used as the Compute server's pod name prefix. However, because the job name becomes the pod name, the job name must meet Kubernetes

pod name criteria. Therefore, the characters in the job name that fail the criteria are converted to meet the criteria.

Note:

- The maximum length of the prefix is set to 100. The prefix naming convention must follow the regex set by Kubernetes: `[a-z0-9]([-a-z0-9]*[a-z0-9])?(\. [a-z0-9]([-a-z0-9]*[a-z0-9])?)*`. For more information, see [Object Names and IDs](#).
 - If a reusable Compute server is in use, the job name is unused. The Compute server pod is already named. See [Configure Reusable Compute Servers](#) and “Reusable Compute Server” in *SAS Viya Platform: Programming Run-Time Servers*.
 - Compute provider (and Compute service) might not create a new Compute server. For details, see “Scenarios When Compute Server Is Not Created” on page 61.
-

Here are some examples of conversions:

- Spaces are converted to hyphens. Upper case characters are converted to lower case. A required trailing hyphen is added.
My Job Name is converted to my-job-name-
 - Spaces are converted to hyphens. Upper case characters are converted to lower case. Special characters are removed. A required trailing hyphen is added.
HR & \$\$ Processing is converted to hr-processing-
 - If a job name is too long, it is truncated to the length limit.
 - If a job name contains a mixture of ASCII and non-ASCII characters, the non-ASCII characters are removed from the job name.
 - If a job name contains all non-ASCII characters, the job name becomes sas-compute-server-.
 - If all of the failing characters are removed and the job name is still invalid, the job name becomes sas-compute-server-.
- (#)&(\$) is converted to - and then to sas-compute-server-.

Scenarios When Compute Server Is Not Created

There are two scenarios when Compute service (or Compute provider) does not create a Compute server.

A Reusable Compute Server Is Specified in Compute Context

Compute service does not create a Compute server if these criteria are met:

- The call to Compute service (or Compute provider) specifies a Compute context that uses a reusable Compute server.
- An idle reusable Compute server is available.

When both of these criteria are met, the Compute service creates a session on the idle server and no new server is started.

See [Configure Reusable Compute Servers](#) and [“Reusable Compute Server” in SAS Viya Platform: Programming Run-Time Servers](#).

A Compute Session ID Is Provided in the Job Request

When submitting a Job Execution Service (JES) job request, the user specifies a Compute session ID instead of a Compute context. In this scenario, the user has created the Compute session. Compute provider merely submits the job to that session. Compute provider does not create a new server.

For concepts, see [“How Jobs and Flows Work” in SAS Viya Platform: Jobs and Flows](#).